

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Construyendo Agentes de Inteligencia Artificial

Día 1 : Agentes - Fundamentos y Modelos de Lenguaje

Lunes 27 de Julio del 2026

Sobre este Curso

- La materia propone una mirada sobre los Agentes de Inteligencia Artificial.
- 5 clases teórica-prácticas y evaluación en la última clase
- Aprobación:
 - Evaluación es un trabajo en clase - Viernes 31 de Julio del 2026
- Metodología de trabajo:
 - teórico-práctico con actividades en formas de Agentes y Notebooks para entender con código los temas más fundamentales
 - es un espacio para compartir las experiencias profesionales, acompañadas por reflexión y pensamiento de oportunidades futuras.
 - preguntas para practicar para el examen clase a clase.
- Material en inglés y español

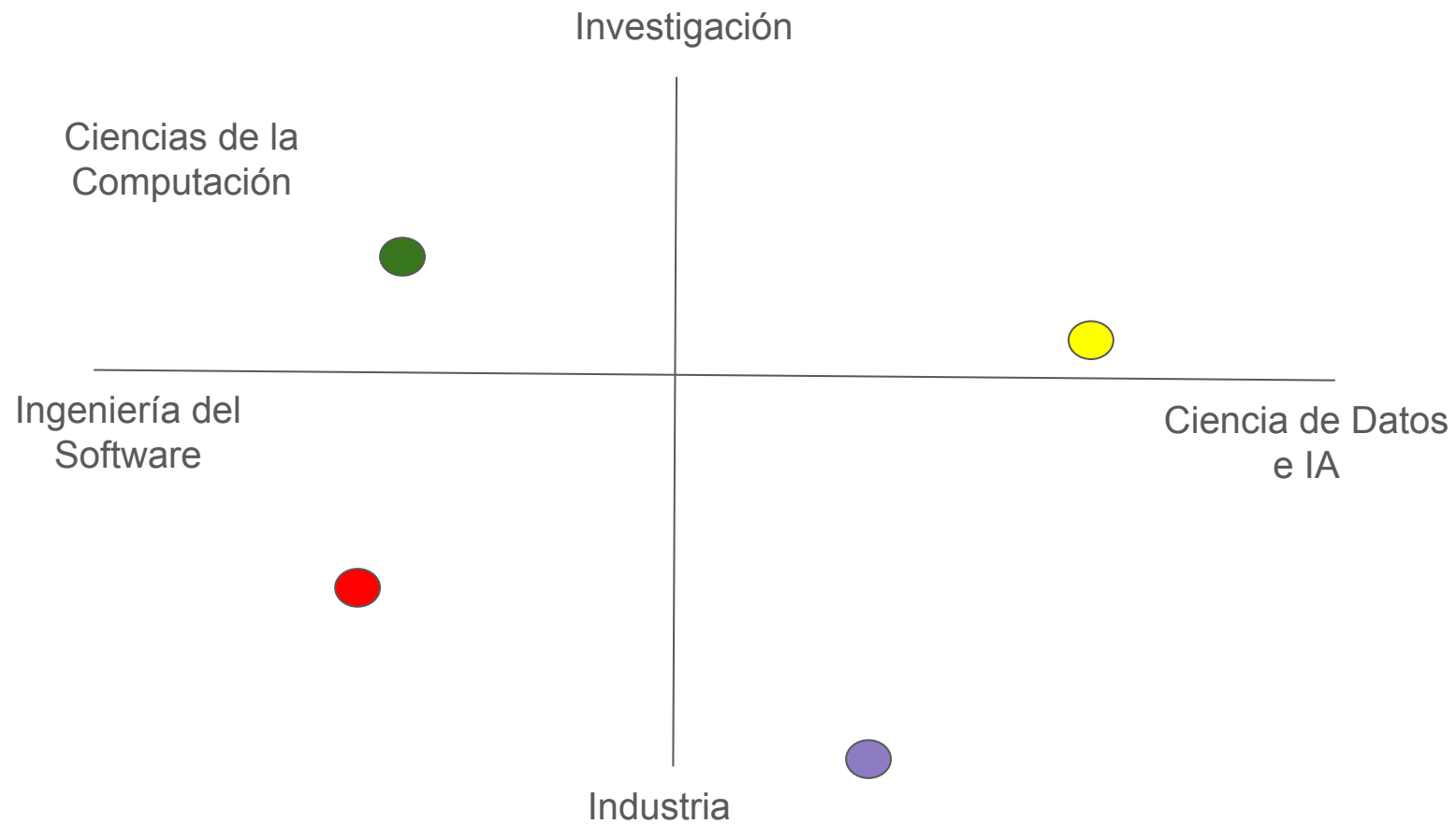


<http://agentes-ia-eci2026.letyrodri.com>

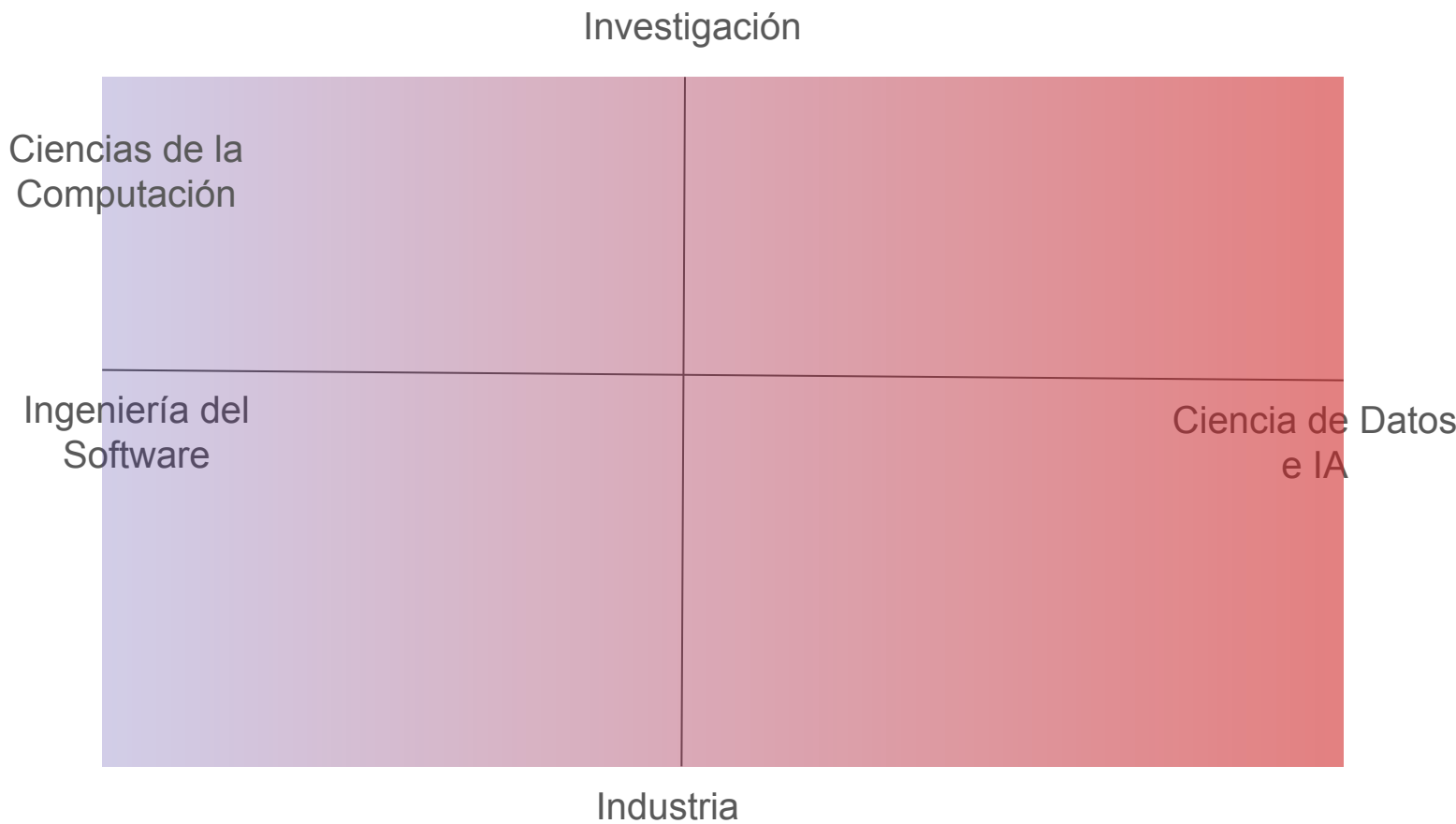
Calendario del Curso

- Lunes 27/7 - Clase 1 - Agentes: Fundamentos y Modelos de Lenguajes
- Martes 28/7 - Clase 2 - Diseñando Agentes
- Miércoles 29/7 - Clase 3 - Agentes - Frameworks y Google ADK
- Jueves 30/7 - Clase 4 - Construyendo Agentes RAG y Basados en Tareas
- Viernes 31/7 - Clase 5 - Multi-agentes, Evaluación y Futuro
- Viernes 31/7 - Evaluación - 19:50 hs.

Investigación vs. Industria - Cs. de Datos vs. Ing. del Software



Investigación vs. Industria - Cs. de Datos vs. Ing. del Software



DIA 1

Fundamentos y Modelos de Lenguaje

Agentes vs. Modelo de Lenguaje

LLM Puro

Depende solo de su entrenamiento interno.



Agente

Bucle de razonamiento con memoria y uso de herramientas.



¿Qué es un Agente?

Perspectiva	Fuente Principal	Definición en una frase
Clásica	Russell & Norvig (1995)	Un agente de IA es una entidad artificial que percibe su entorno y procesa dicha información para tomar decisiones y ejecutar acciones que maximicen las probabilidades de alcanzar un objetivo específico.
Arquitectónica	Wang et al. (Survey)	Un "Cerebro" (LLM) conectado a Memoria, Planificación y Herramientas.
Operativa	Andrew Ng	Un flujo de trabajo donde la IA itera, se autocorrige y usa herramientas externas.

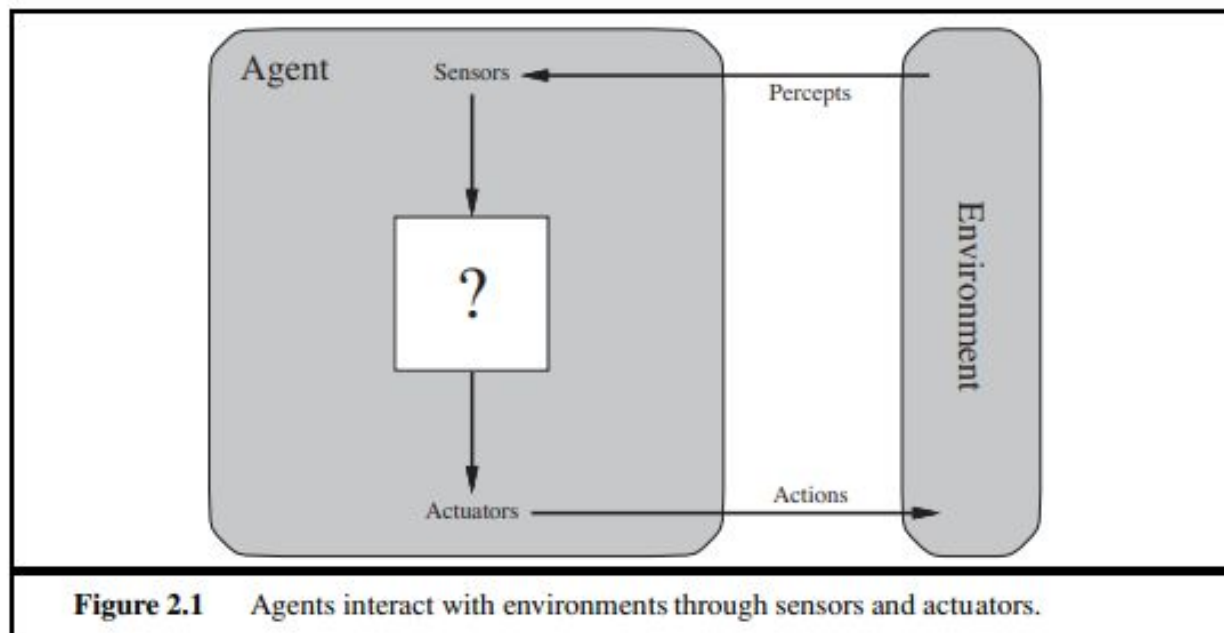
The Rise and Potential of Large Language Model Based Agents: A Survey

<https://arxiv.org/pdf/2309.07864>

Agentes



Agentes de IA



Libro: Artificial Intelligence: A modern approach - Norvig & Rusell

Primeras ideas: LLM+Memoria

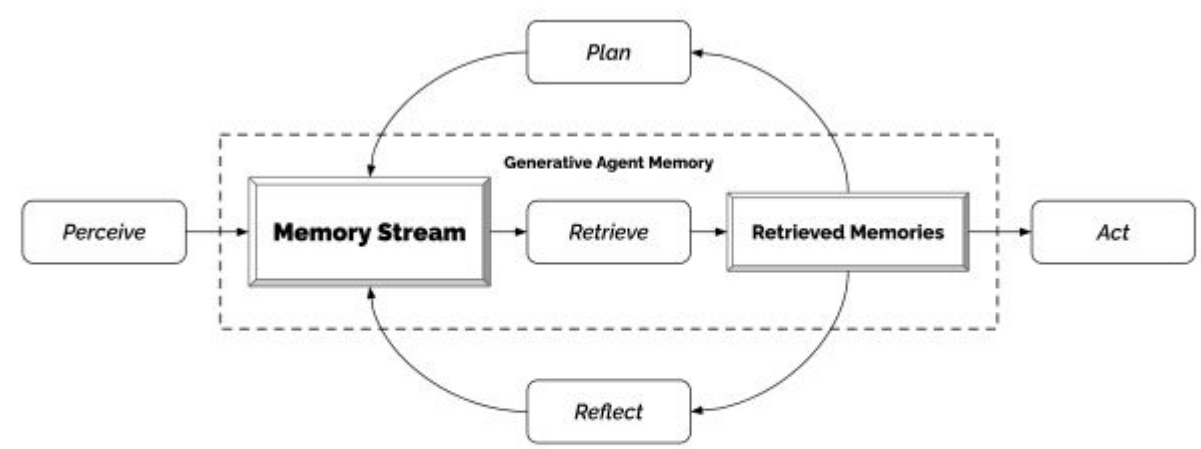


Figure 5: Our generative agent architecture. Agents perceive their environment, and all perceptions are saved in a comprehensive record of the agent's experiences called the memory stream. Based on their perceptions, the architecture retrieves relevant memories and uses those retrieved actions to determine an action. These retrieved memories are also used to form longer-term plans and create higher-level reflections, both of which are entered into the memory stream for future use.

Idea que aparece en un Paper: <https://arxiv.org/pdf/2304.03442>

Primeras ideas: LLM+Tools

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Figure 1: Exemplary predictions of Toolformer. The model autonomously decides to call different APIs (from top to bottom: a question answering system, a calculator, a machine translation system, and a Wikipedia search engine) to obtain information that is useful for completing a piece of text.

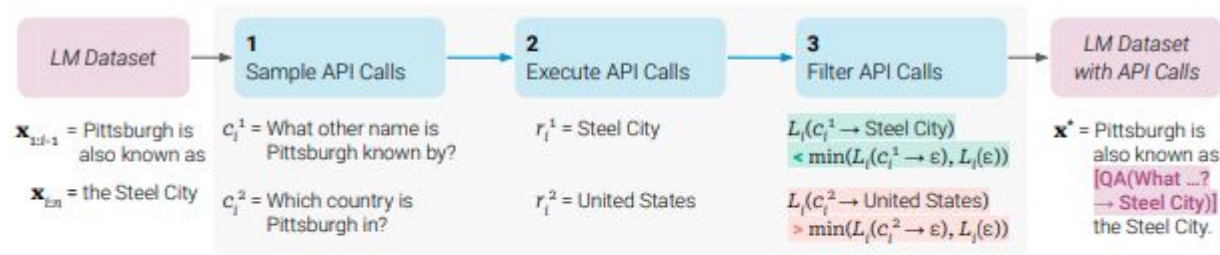


Figure 2: Key steps in our approach, illustrated for a *question answering* tool: Given an input text $\mathbf{x}$, we first sample a position i and corresponding API call candidates $c_i^1, c_i^2, \dots, c_i^k$. We then execute these API calls and filter out all calls which do not reduce the loss L_i over the next tokens. All remaining API calls are interleaved with the original text, resulting in a new text $\mathbf{x}^*$.

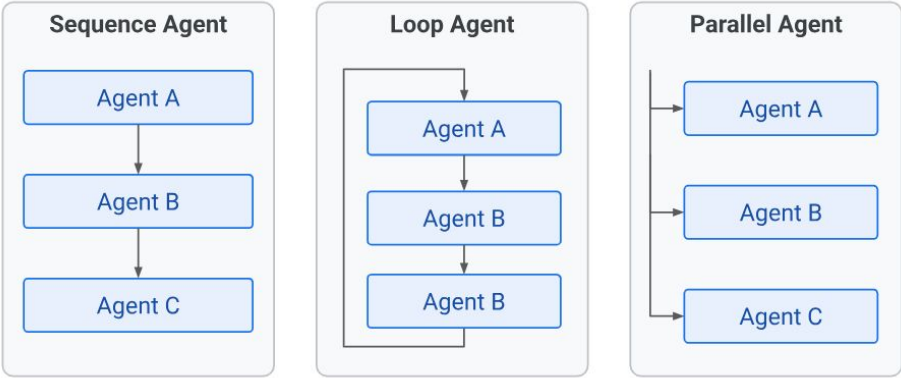
Idea que aparece en un Paper: <https://arxiv.org/pdf/2302.04761>

Diferencia: Flujos de trabajo automatizados (Automated Workflows)

También, le dicen AI orchertration. Se trata de una secuencia de tareas que son determinísticas. Se puede pensar como una secuencia de pasos if-then-else. Es rigido, siguiendo un camino y no usan (a diferencia del agente de LLM) la razón ni planean dinámicamente o se adaptan a las circunstancias no establecidas anteriormente.

Veremos **Google ADK** implementa algo similar que une los distintos subagentes:

Template workflows



Los agentes de flujo de trabajo de plantilla operan basados en una lógica predefinida. Determinan la secuencia de ejecución según su tipo, como secuencial, paralelo o de bucle, sin consultar a un modelo de IA para obtener asistencia con la orquestación. Este enfoque da como resultado patrones de ejecución deterministas y predecibles.

Diferencia entre LLM vs Flujos automatizados de trabajo vs Agente de IA

Características	LLM	Flujos automatizados	AI Agent
Función	Generar texto, imágenes, videos a partir de un prompt	Ejecutan tareas predefinidas y estáticas bajo condiciones	De forma autónoma busca lograr un gol
Decisiones	Generación probabilística del próximo token	Basado en if-then-else harcodeados	Reasona, planea, y toma decisiones dinámicas para manejar problemas complejos
Memoria y Estado	Stateless: cada iteración es nueva (sin memoria)	Generalmente es stateless	Stateful: mantiene memorias short-term y long-term que las usa para responder o aprender
Adaptabilidad	No se adapta. Solo con fine-tuning pero sigue siendo estático.	Rigido: cualquier cambio require programación	Altamente adaptable, aprende de experiencias y reacciona a cambios del entorno
Modelo de interacción	Recibe un prompt, da la respuesta	Disparados por un evento, ejecuta un script fijo	Opera continuamente es sensor-reasonar-planear-actua para conseguir su gol
Manejo de fallos	Pasivo. Puede alucinar sin darse cuenta del fallo	Rigido: Si falla da error o ejecuta algún camino predeterminado de manejo del error	Resiliente/Autocorrije: puede detectar errores, reflexionar sobre la cuasa y actuar autonomament reintentando con otra estrategia o herramienta.

Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

Diferencia entre LLM vs Flujos automatizados de trabajo vs Agente de IA

Características	LLM	Flujos automatizados	AI Agent
Función	Generar texto, imágenes, videos a partir de un prompt	Ejecutan tareas predefinidas y estáticas bajo condiciones	De forma autónoma busca lograr un gol
Decisiones	Generación probabilística del próximo token	Basado en if-then-else harcodeados	Reasona, planea, y toma decisiones dinámicas para manejar problemas complejos
Memoria y Estado	Stateless: cada iteración es nueva (sin memoria)	Generalmente es stateless	Stateful: mantiene memorias short-term y long-term que las usa para responder o aprender
Adaptabilidad	No se adapta. Solo con fine-tuning pero sigue siendo estático.	Rigido: cualquier cambio require programación	Altamente adaptable, aprende de experiencias y reacciona a cambios del entorno
Modelo de interacción	Recibe un prompt, da la respuesta	Disparados por un evento, ejecuta un script fijo	Opera continuamente es sensor-reasonar-planear-actua para conseguir su gol
Manejo de fallos	Pasivo. Puede alucinar sin darse cuenta del fallo	Rigido: Si falla da error o ejecuta algún camino predeterminado de manejo del error	Resiliente/Autocorrije: puede detectar errores, reflexionar sobre la cuasa y actuar autonomament reintentando con otra estrategia o herramienta.

Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

Diferencia entre LLM vs Flujos automatizados de trabajo vs Agente de IA

Características	LLM	Flujos automatizados	AI Agent
Función	Generar texto, imágenes, videos a partir de un prompt	Ejecutan tareas predefinidas y estáticas bajo condiciones	De forma autónoma busca lograr un gol
Decisiones	Generación probabilística del próximo token	Basado en if-then-else harcodeados	Reasona, planea, y toma decisiones dinámicas para manejar problemas complejos
Memoria y Estado	Stateless: cada iteración es nueva (sin memoria)	Generalmente es stateless	Stateful: mantiene memorias short-term y long-term que las usa para responder o aprender
Adaptabilidad	No se adapta. Solo con fine-tuning pero sigue siendo estático.	Rigido: cualquier cambio require programación	Altamente adaptable, aprende de experiencias y reacciona a cambios del entorno
Modelo de interacción	Recibe un prompt, da la respuesta	Disparados por un evento, ejecuta un script fijo	Opera continuamente es sensor-reasonar-planear-actua para conseguir su gol
Manejo de fallos	Pasivo. Puede alucinar sin darse cuenta del fallo	Rigido: Si falla da error o ejecuta algún camino predeterminado de manejo del error	Resiliente/Autocorrije: puede detectar errores, reflexionar sobre la cuasa y actuar autonomament reintentando con otra estrategia o herramienta.

Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

Diferencia entre LLM vs Flujos automatizados de trabajo vs Agente de IA

Características	LLM	Flujos automatizados	AI Agent
Función	Generar texto, imágenes, videos a partir de un prompt	Ejecutan tareas predefinidas y estáticas bajo condiciones	De forma autónoma busca lograr un gol
Decisiones	Generación probabilística del próximo token	Basado en if-then-else harcodeados	Reasona, planea, y toma decisiones dinámicas para manejar problemas complejos
Memoria y Estado	Stateless: cada iteración es nueva (sin memoria)	Generalmente es stateless	Stateful: mantiene memorias short-term y long-term que las usa para responder o aprender
Adaptabilidad	No se adapta. Solo con fine-tuning pero sigue siendo estático.	Rigido: cualquier cambio require programación	Altamente adaptable, aprende de experiencias y reacciona a cambios del entorno
Modelo de interacción	Recibe un prompt, da la respuesta	Disparados por un evento, ejecuta un script fijo	Opera continuamente es sensor-reasonar-planear-actua para conseguir su gol
Manejo de fallos	Pasivo. Puede alucinar sin darse cuenta del fallo	Rigido: Si falla da error o ejecuta algún camino predeterminado de manejo del error	Resiliente/Autocorrije: puede detectar errores, reflexionar sobre la cuasa y actuar autonomament reintentando con otra estrategia o herramienta.

Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

Diferencia entre LLM vs Flujos automatizados de trabajo vs Agente de IA

Características	LLM	Flujos automatizados	AI Agent
Función	Generar texto, imágenes, videos a partir de un prompt	Ejecutan tareas predefinidas y estáticas bajo condiciones	De forma autónoma busca lograr un gol
Decisiones	Generación probabilística del próximo token	Basado en if-then-else harcodeados	Reasona, planea, y toma decisiones dinámicas para manejar problemas complejos
Memoria y Estado	Stateless: cada iteración es nueva (sin memoria)	Generalmente es stateless	Stateful: mantiene memorias short-term y long-term que las usa para responder o aprender
Adaptabilidad	No se adapta. Solo con fine-tuning pero sigue siendo estático.	Rigido: cualquier cambio require programación	Altamente adaptable, aprende de experiencias y reacciona a cambios del entorno
Modelo de interacción	Recibe un prompt, da la respuesta	Disparados por un evento, ejecuta un script fijo	Opera continuamente es sensor-reasonar-planear-actua para conseguir su gol
Manejo de fallos	Pasivo. Puede alucinar sin darse cuenta del fallo	Rigido: Si falla da error o ejecuta algún camino predeterminado de manejo del error	Resiliente/Autocorrije: puede detectar errores, reflexionar sobre la cuasa y actuar autonomament reintentando con otra estrategia o herramienta.

Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

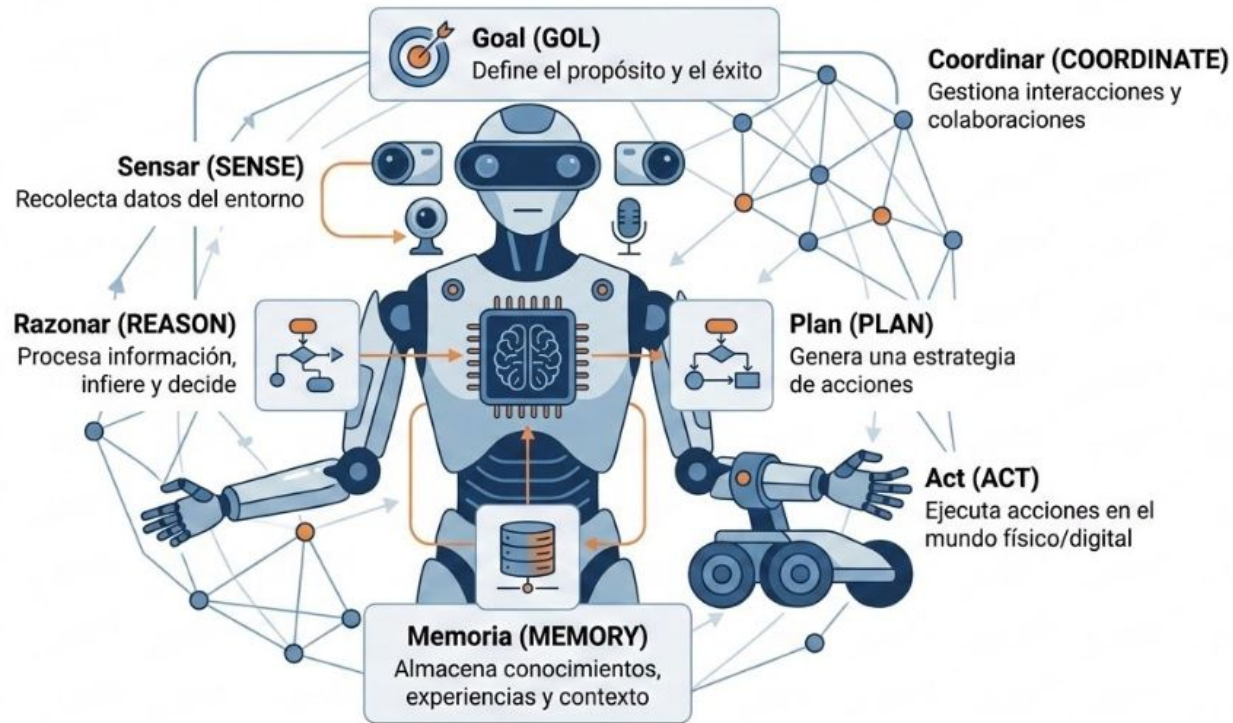
Diferencia entre LLM vs Flujos automatizados de trabajo vs Agente de IA

Características	LLM	Flujos automatizados	AI Agent
Función	Generar texto, imágenes, videos a partir de un prompt	Ejecutan tareas predefinidas y estáticas bajo condiciones	De forma autónoma busca lograr un gol
Decisiones	Generación probabilística del próximo token	Basado en if-then-else harcodeados	Reasona, planea, y toma decisiones dinámicas para manejar problemas complejos
Memoria y Estado	Stateless: cada iteración es nueva (sin memoria)	Generalmente es stateless	Stateful: mantiene memorias short-term y long-term que las usa para responder o aprender
Adaptabilidad	No se adapta. Solo con fine-tuning pero sigue siendo estático.	Rigido: cualquier cambio require programación	Altamente adaptable, aprende de experiencias y reacciona a cambios del entorno
Modelo de interacción	Recibe un prompt, da la respuesta	Disparados por un evento, ejecuta un script fijo	Opera continuamente es sensor-reasonar-planear-actua para conseguir su gol
Manejo de fallos	Pasivo. Puede alucinar sin darse cuenta del fallo	Rigido: Si falla da error o ejecuta algún camino predeterminado de manejo del error	Resiliente/Autocorrije: puede detectar errores, reflexionar sobre la cuasa y actuar autonomament reintentando con otra estrategia o herramienta.

Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

El ciclo del Agente

Los agentes sensan el entorno, razon sobre nueva información en el contexto con su gol y memoria, formula un plan y actua sobre el entorno



Libro: "Agentic Architectural Patterns for Building Multi-Agent Systems" Dr. Ali Arsanjani y Juan Pablo Bustos

Ejemplo: Agente de viajes

Gol

Es un Agente para viajes que debe cumplir los pedidos del usuario relacionados con reserva de vuelos y alojamiento

Sensar

Recibir los pedidos de los usuarios en Lenguaje Natural

“Necesito reservar un vuelo a Chile”

Razonar

Analizar el pedido, detectar la intención, ver si la información es suficiente

“Reservar vuelo”

Planear

Basado en lo que razonó hace un plan:

1. Analizar que información falta (fecha ida, regreso, personas, destino, origen)
2. Solicitar información faltante
3. Si la información esta completa, buscar los vuelos disponibles.

Actuar

El agente ejecuta el plan y llama a las herramientas necesarias.

“Una vez que tiene la información necesaria llama a la herrameinta dde reserva de vuelos (API)”

Memoria

Corto Plazo

Preferencias del pedido del usuario: fechas, destino, origen, cantidad de pasajeros

Largo Plazo

Prefencias de destinos, cantidad de vuelos pedidos, nombres de los pasajeros frecuentes

Coordinar: La implementación puede tener varios subagentes especializados: Agendar Vuelos - Agendar Hoteles

Demo

Un Agente de IA en acción

Antes hablemos de

Modelos de Lenguaje

Elementos del Agente: Modelo

El modelo va a permitir al agente:

- interactuemos por voz o por chat con el usuario (si es interactivo)
- planificar y realizar las tareas que requiera el usuario o para las que esté diseñado
- generar texto, código, imágenes, audio o video según corresponda
- hacer verificaciones sobre salidas u otros resultados internos
- y mucho más.

Se lo puede ver como el cerebro del agente y puede haber varios modelos, de distinta naturaleza pero en general, para que el agente sea conversacional, va a tener al menos una LLM o MLLM.

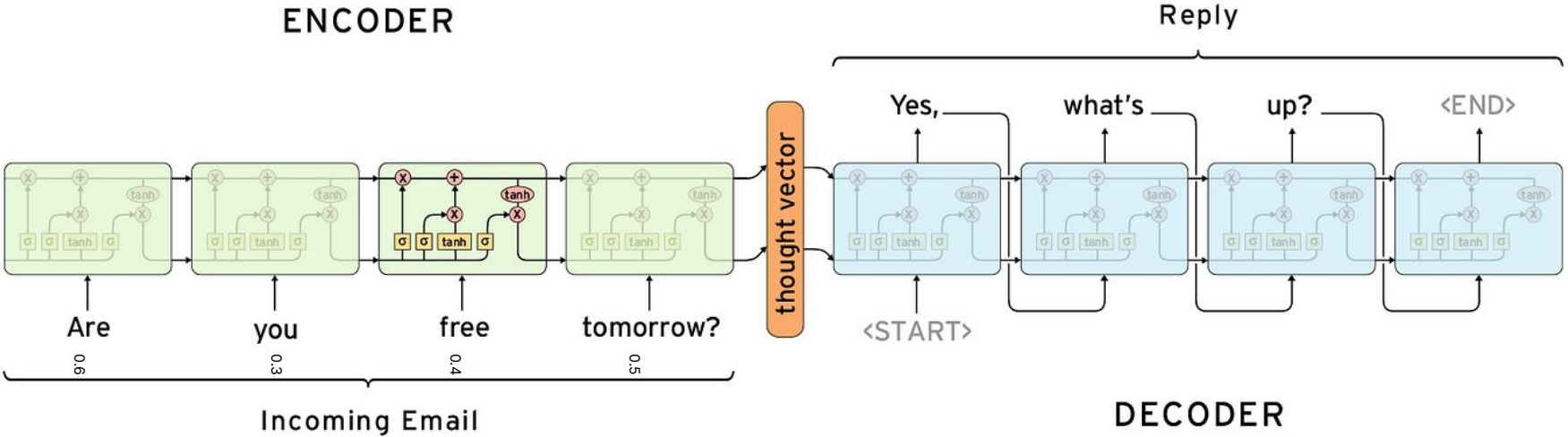
Recuerden que la palabra agente es amplia, podría no ser incluso conversacional, pero en este curso nos estamos enfocando principalmente a aquellos que usan LLMs.



Modelos de lenguaje y su origen

- Los modelos de lenguaje son modelos de Inteligencia Artificial **probabilísticos** que a partir de una entrada de texto se predice la siguiente palabra
- Desde el primer modelo Bag of Words en 1954, se ha ido mejorando la tecnología.
- Se creó un método de usar “embeddings”, en el 2013: Word2Vec. Convierte cada palabra en un vector numéricos que capturado la relación semántica entre los vectores de las oraciones
- Siguieron por Recurrent Neural Networks, RNN. Mejoradas con LSTM y RNN Bidireccionales.
- En 2014, surgen las Encoder-Decoder RNN

Arquitectura de Red Neuronal: Encoder-Decoder RNN

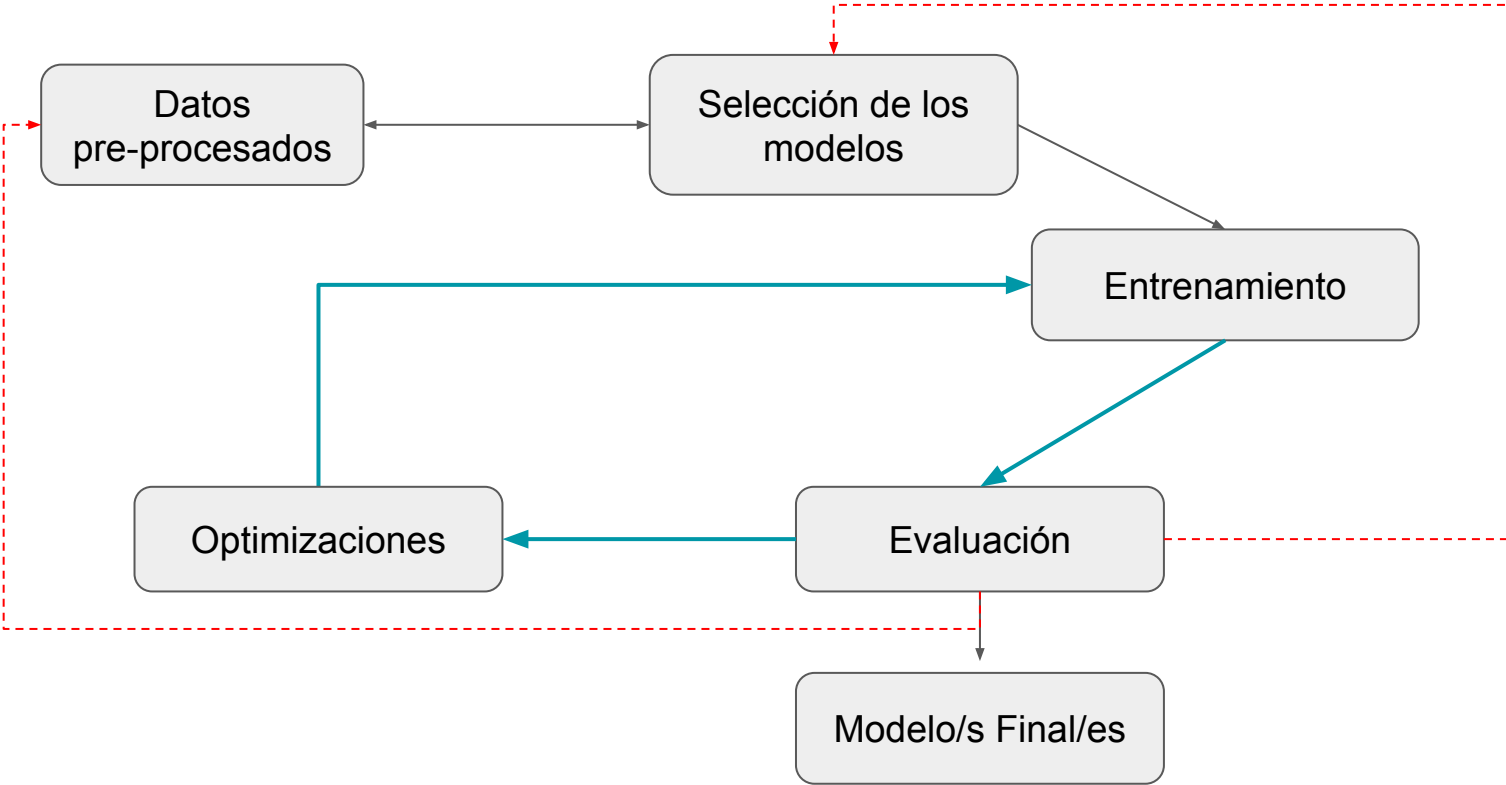


Sequence to Sequence Learning with Neural Networks <https://arxiv.org/pdf/1409.3215>

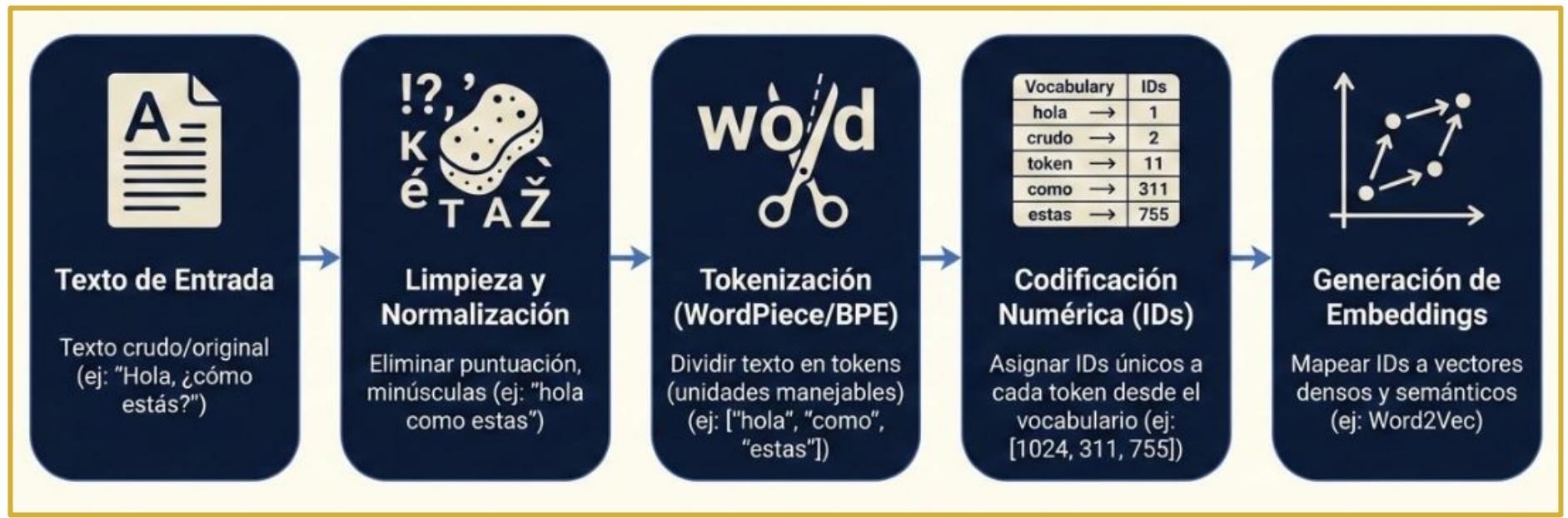
Modelos de Lenguaje actuales: LARGE Language Models (LLMs)

- Están basados en la arquitectura **Transformers**
- Son una respuesta a los problemas que habia con el modelo Sequence to Sequence:
 - La respuesta se generaba en base al último hidden state
 - Tanto para la entrada como la salida, el procesamiento era secuencial y por lo tanto, lento para cadenas largas
- Pueden ser **encoder only, decoder only, encoder-decoder**
- Las grandes empresas como Google, Meta, Amazon, SpaceX pusieron a entrenar estos modelos de arquitectura "Transformer" en basta cantidad de datos.
- Nuevas empresas: Anthropic, OpenAI, Perplexity

¿Cómo se desarrolla un modelo?



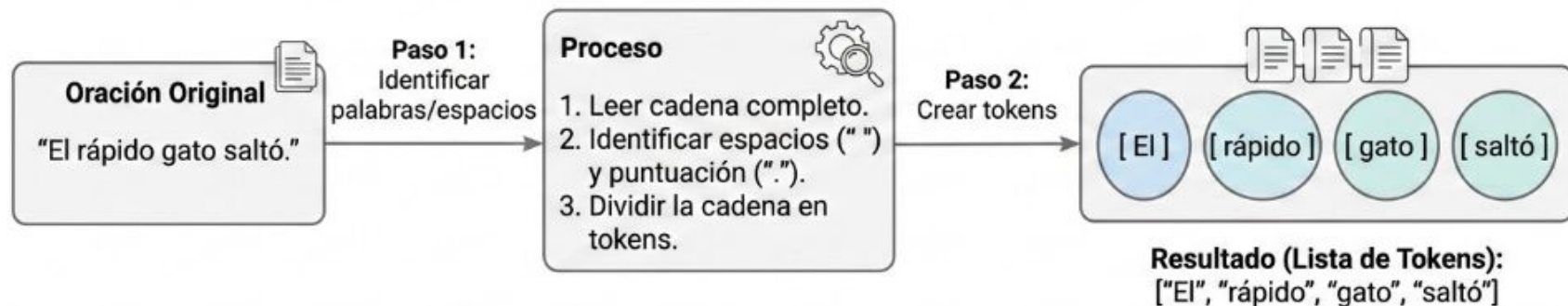
Pre-Procesamiento del Corpus de Entrenamiento



Tokenización

El proceso fundamental en NLP (Procesamiento de Lenguaje Natural) para dividir el texto en unidades manejables llamadas *tokens*.

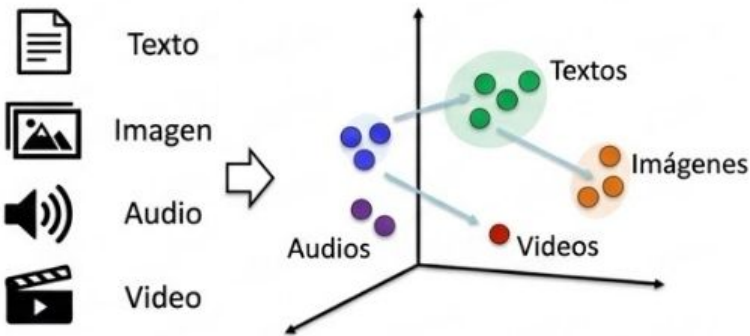
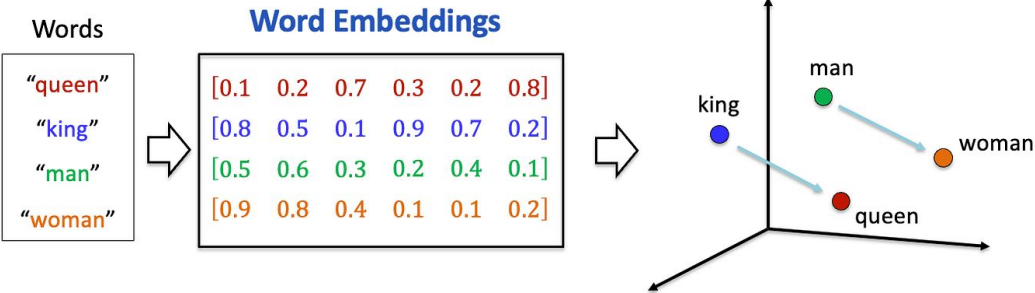
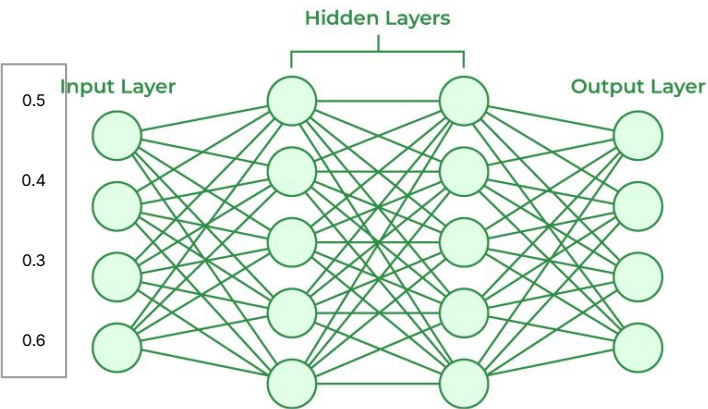
- **Por Caracteres:** Divide el texto por cada letra/carácter.
- **Por Palabras:** Divide el texto por espacios y puntuación.
- **Subpalabras (Byte-Pair Encoding, BPE):** Algoritmo moderno y eficiente que divide palabras infrecuentes en subunidades más comunes.



https://huggingface.co/docs/transformers/tokenizer_summary

Embeddings

En NLP, las palabras pueden ser representadas por vectores numérico densos que capturan la similaridad semántica entre ellas.



Los Embeddings son vectores densos que codifican la información permitiendo alguna relación de distancia entre lo que representan. Pueden encodear texto, imágenes, etc.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

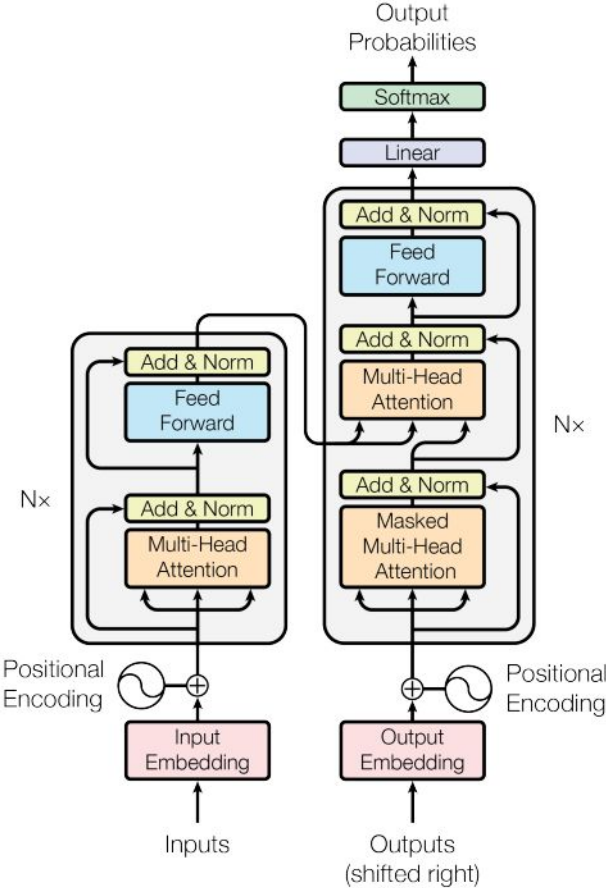


Figure 1: The Transformer - model architecture.

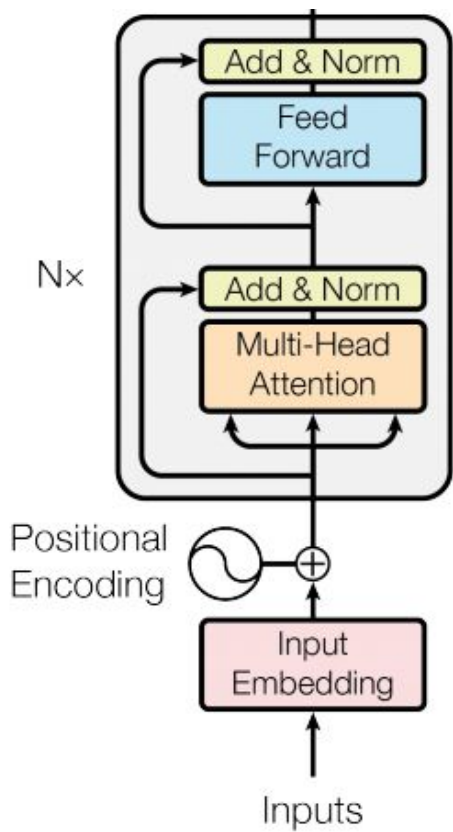
Arquitectura Transformers: BERT

BERT es un modelo encoder-only. Consiste en apilar varias capas encoder del modelo transformers. Como no está conectado a un decoder, sólo puede ser usado para encontrar el vector encoding de la entrada.

Son 12 bloques encoder apilados. Output se lo pasa por un maxpooling layer y fully connected layer seguido por una función de activación no lineal (ReLU, Tanh).

Se le puede agregar estas capas para otras tareas como haga una clasificación y prediga etiquetas o para evaluar similaridad.

Por ejemplo, para Análisis de Sentimientos (Sentiment Analysis).



Arquitectura Transformers: GPT-1

GPT-1 es un modelo **decoder-only**. Se focalizan en predecir el próximo token.

Estos modelos pre-entrenados pueden ejecutar tareas como clasificación, summarización, traducción, por mencionar algunas de sus tantas aplicaciones.

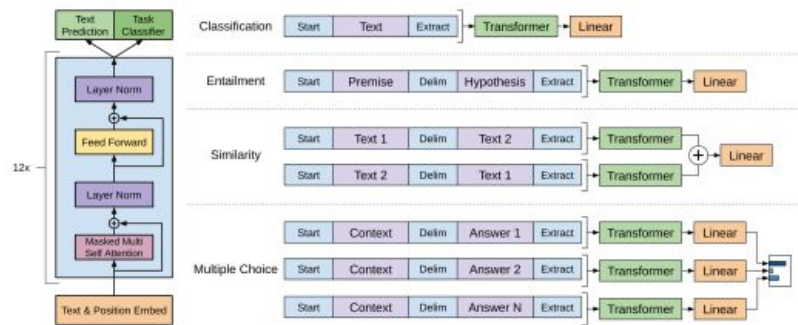


Figure 1: (left) Transformer architecture and training objectives used in this work. (right) Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Generative Pre-trained Transformers:
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

Masked Self-Attention

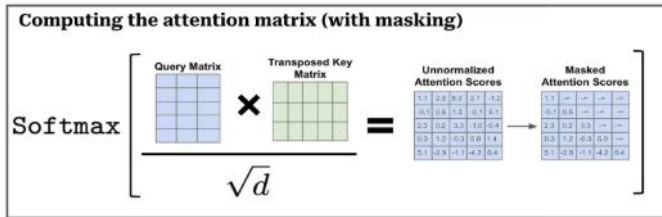
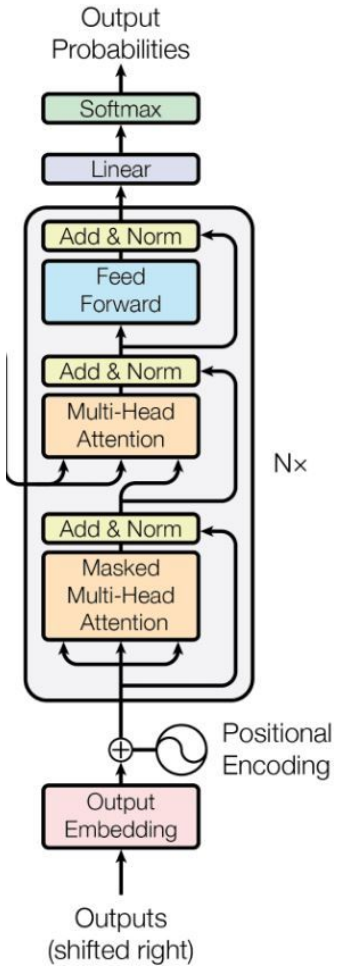
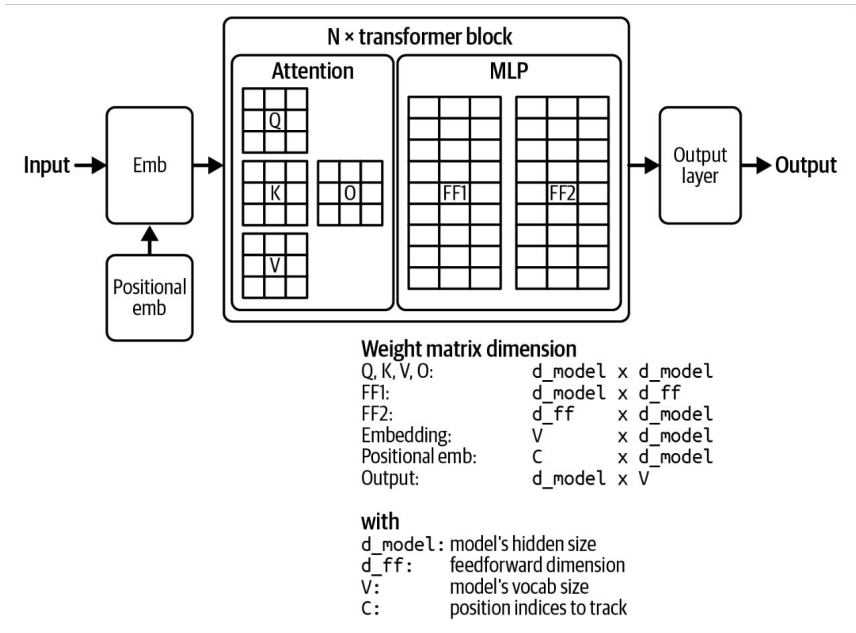


Imagen de:
<https://stackoverflow.blog/2024/09/26/masked-self-attention-how-lms-learn-relationships-between-tokens/>



Arquitectura Transformer: Cantidad de Parámetros



Generación y Modelo	Bloques Transformer (num_hidden_layers)	Dimensión del Modelo (hidden_size)	Dimensión FeedForward (intermediate_size)	Tamaño del Vocabulario (vocab_size)	Longitud de Contexto (max_position_embeddings)
LLaMA 2 (7B)	32	4096	11008	32K	4K
LLaMA 2 (13B)	40	5120	13824	32K	4K
LLaMA 2 (70B)	80	8192	28672	32K	4K
LLaMA 3 (8B)	32	4096	14336	128K	8K
LLaMA 3 (70B)	80	8192	28672	128K	8K
LLaMA 3.1 (8B)	32	4096	14336	128K	128K
LLaMA 3.1 (70B)	80	8192	28672	128K	128K
LLaMA 3.1 (405B)	126	16384	53248	128K	128K

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

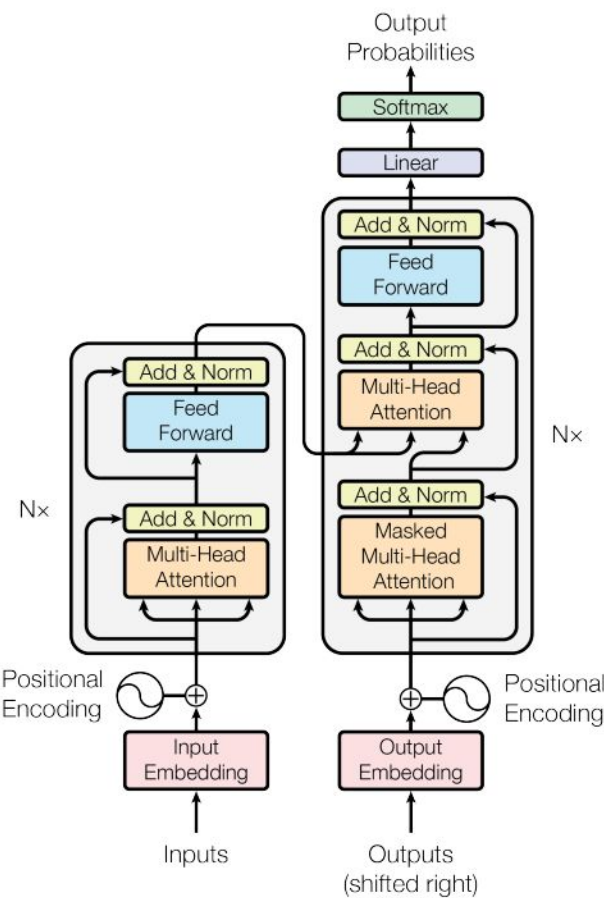


Figure 1: The Transformer - model architecture.

Entendiendo la arquitectura Transformer: Cantidad de Parámetros

La tarjeta gráfica más potente y estándar de la industria para IA (la NVIDIA H100) tiene **80 GB de VRAM**. El modelo más chico de Llama 2 tiene 7 billones de parámetros.

$7,000,000,000 \text{ parámetros} \times 2 \text{ bytes} = 14,000,000,000 \text{ bytes} = \mathbf{14 \text{ GB}}$

Precisión Completa/Media (FP16 o BF16): Es como sale el modelo del horno. Cada parámetro ocupa **2 bytes**.

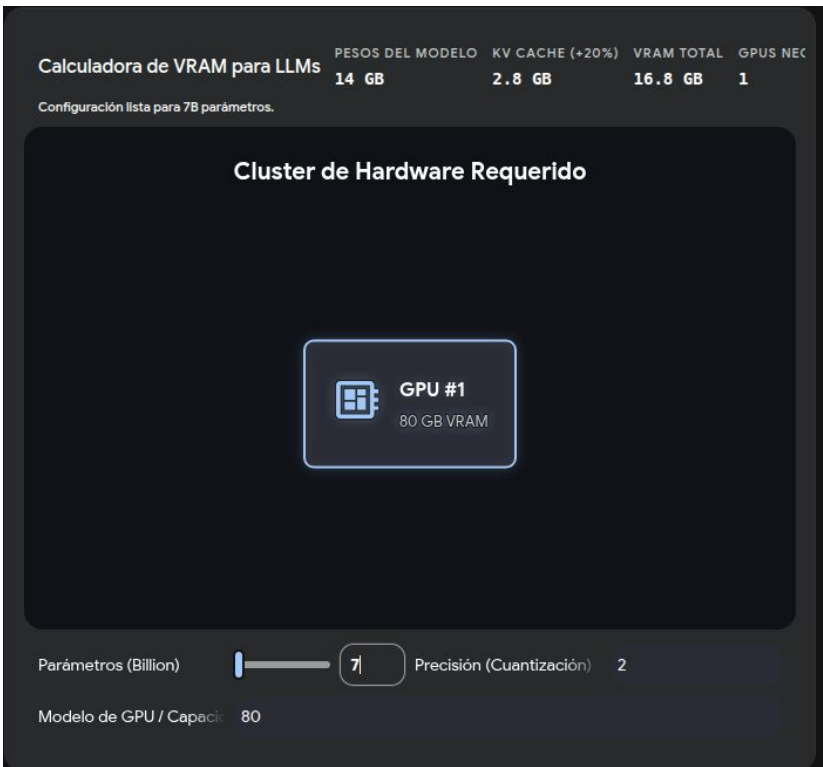
- $7 \text{ billones} \times 2 \text{ bytes} = \mathbf{\sim 14 \text{ GB de VRAM}}$.

Cuantización a 8-bit (INT8): Se "comprimen" los decimales perdiendo un mínimo de precisión. Cada parámetro ocupa **1 byte**.

- $7 \text{ billones} \times 1 \text{ byte} = \mathbf{\sim 3.5 \text{ GB de VRAM}}$.

Cuantización a 4-bit (INT4 / AWQ): Compresión agresiva muy popular para correr modelos en casa. Cada parámetro ocupa **0.5 bytes**.

- $7 \text{ billones} \times 0.5 \text{ bytes} = \mathbf{\sim 3.37 \text{ GB de VRAM}}$.



Entendiendo la arquitectura Transformer: Cantidad de Parámetros

La tarjeta gráfica más potente y estándar de la industria para IA (la NVIDIA H100) tiene **80 GB de VRAM**. El modelo más grande de Llama tiene 405.8 billones

Precisión Completa/Media (FP16 o BF16): Es como sale el modelo del horno. Cada parámetro ocupa **2 bytes**.

- $405.8 \text{ billones} \times 2 \text{ bytes} = \sim \mathbf{811 \text{ GB de VRAM}}$.

Cuantización a 8-bit (INT8): Se "comprimen" los decimales perdiendo un mínimo de precisión. Cada parámetro ocupa **1 byte**.

- $405.8 \text{ billones} \times 1 \text{ byte} = \sim \mathbf{405 \text{ GB de VRAM}}$.

Cuantización a 4-bit (INT4 / AWQ): Compresión agresiva muy popular para correr modelos en casa. Cada parámetro ocupa **0.5 bytes**.

- $405.8 \text{ billones} \times 0.5 \text{ bytes} = \sim \mathbf{202 \text{ GB de VRAM}}$.



Historia de los Modelos de Lenguajes y Modelos de Lenguaje Multimodales basados en arquitectura Transformer

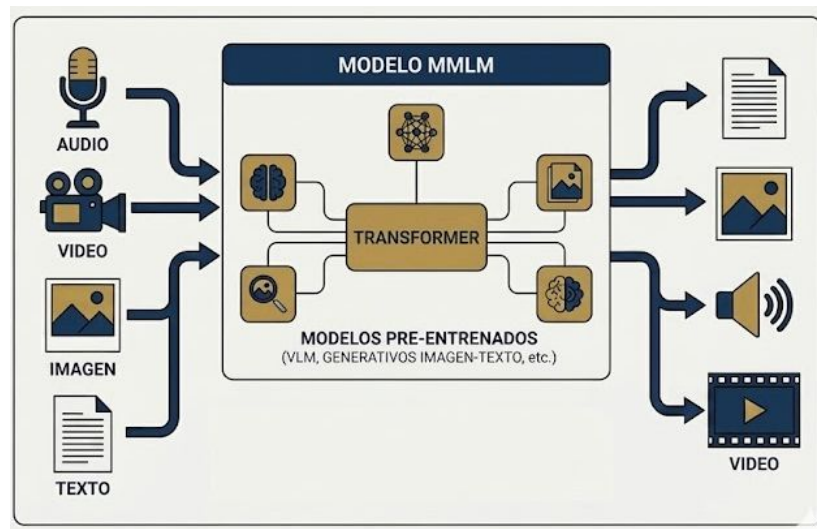


Multimodal Large Language Models (MLLMs)

Los modelos iniciales de transformer eran texto a texto (text-to-text) esto significaba que se escribía una consulta en lenguaje natural, en texto, y respondían texto.

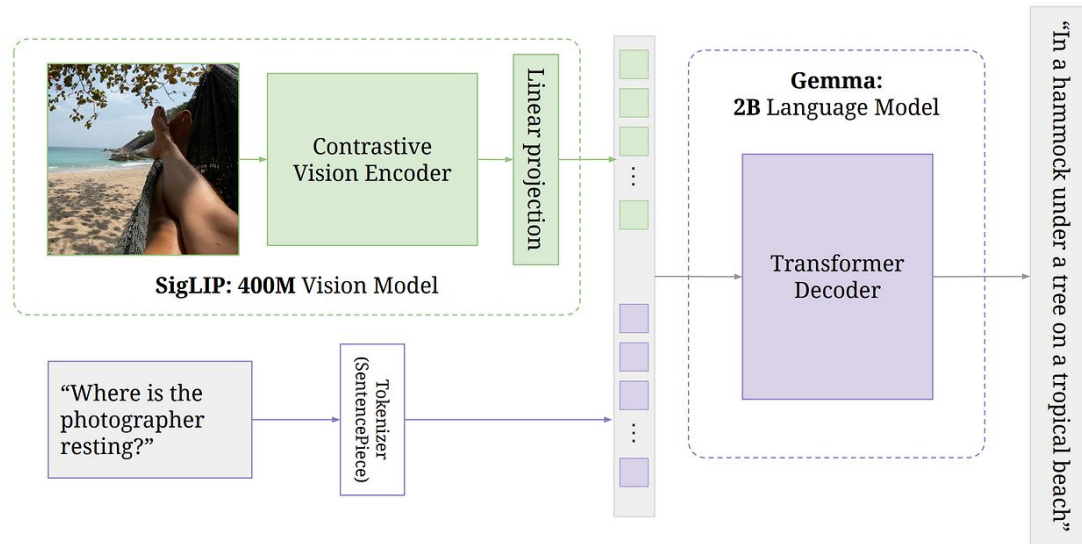
Se decidió escalar estos modelos para que pudieran procesar audio e información visual como videos o imágenes tanto como la entrada y la salida del modelo.

- Los tipos de datos (modalities):
 - texto,
 - imágenes,
 - audio
 - y video.
- Usan varios modelos pre-entrenados: modelos generativos imagen-texto o visión-lenguaje (Vision Language Models VLM)



Cómo se agrega lo visual

- El paper original de Transformers recibe un texto y genera un texto
- Para que sean multimodal hay que combinar esta técnica con modelos pre-entrenados de Visión por Computadora, que trabajen con imágenes o videos de entrada.



PaliGemma's architecture: a SigLIP image encoder feeds into a Gemma decoder LM.

Prompt Engineering

Prompt engineering es la disciplina que crea y optimizar prompts para instruir a los modelos de lenguaje determinados comportamientos y respuestas buscadas

Se busca trasladar nuestras ideas a partir de una conversación regular de forma que el modelo de lenguaje pueda entender específicamente lo que necesitamos

Cuanto más claros y especificos seamos con lo que buscamos, obtendremos una respuesta satisfactoria a partir de un único prompt

Muchas veces se precisa varios prompts para completar una tarea. Prompting lo podemos ver como **un proceso iterativo**

Prompting: recomendaciones generales

- Communicate de forma **clara y precisa**. No te sobrecompliques.
- **Itera rapidamente**. Siéntete comfortable con ir refinando los resultados constantemente.
- Considera los **casos borde**
- Lee las salidas con atención. **Asegurate que el modelo sigue las instrucciones como esperas.**
- **Considera como el modelo podria interpretar tus instrucciones** de forma diferente a la esperada (teoria de la mente)
- **Elimina asumpciones** y communicate claramente con la información completa.
- **Prueba los prompts con imperfecciones**, realistas a las entradas de los usuarios.
- **Trata los prompts como código**. Usa control de versiones y traque los experimentos
- **Considera generar o validar los prompts con una LLM**. Se estricto en editarlos y revisarlos también (**human-in-the-loop**)
- Considera factores como **fuentes de datos, latency y el diseño completo del sistema**

Prompt Engineering: Cada modelo es distinto

Como diferentes modelos pueden responder de forma diferente a un mismo prompt, deben ser optimizados para cada modelo

Asi mismo, dada la naturaleza no deterministica de las LLMs, un prompt puede generar diferentes respuestas

Podemos pedirle a la LLMs que incluso nos responda en algún formato particular (ej. Json), con algún límite de palabras, podemos ser muy creativos como si estuviéramos hablando con un Humano.

Role Prompting

Para dar contexto se puede indicar en el prompt el rol que debe asumir el modelo de lenguaje.

Ejemplo:

Actúa como un **nutricionista deportivo experto** con más de 15 años de experiencia trabajando con atletas olímpicos. Tu tono debe ser motivador, basado en la ciencia, pero fácil de entender.

Tu tarea es explicarle a un joven de 18 años que acaba de empezar en el gimnasio por qué es importante consumir proteínas después de levantar pesas. Dame tres consejos prácticos y económicos para que pueda aplicarlo hoy mismo.

Chain Prompting

Encadenar la salida de un prompt con la entrada del siguiente.

Ejemplo:

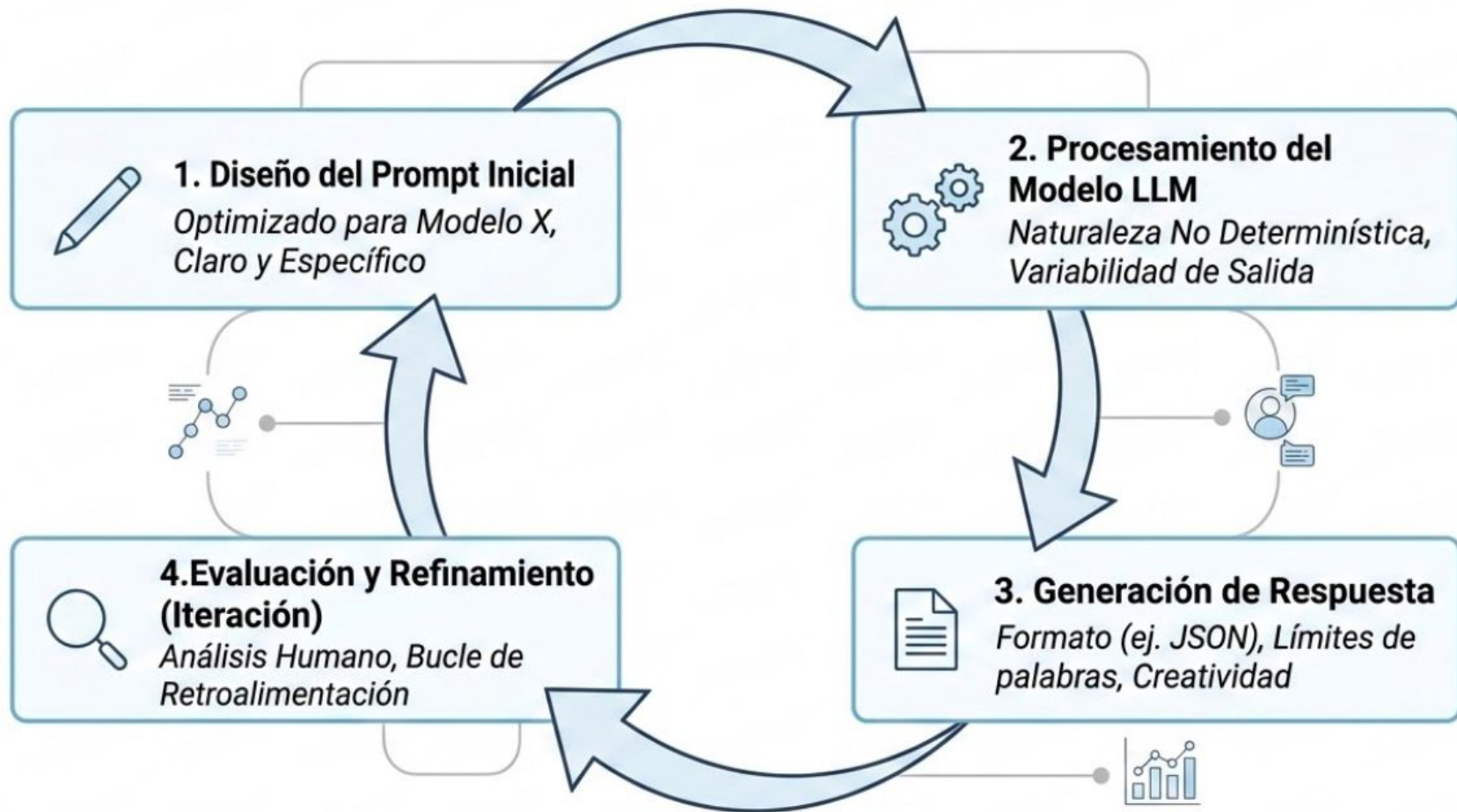
- **Prompt 1:**

"Actúa como un experto en análisis de mercados financieros. Diseña los objetivos y la estructura de una sesión de 90 minutos sobre **el impacto de la inflación y las tasas de interés en los mercados de renta variable**. Divide el contenido en 3 módulos principales y conecta cada uno con casos de estudio macroeconómicos reales."

Tomando como base la estructura generada en el paso anterior, este prompt profundiza en la aplicación técnica y analítica de la sesión.

- **Prompt 2:**

"A partir de la estructura definida, desarrolla el contenido detallado para el segundo módulo, centrado en la evaluación de carteras de inversión y la volatilidad de los activos. Incluye un ejercicio práctico de 15 minutos donde los estudiantes deban aplicar un modelo de análisis de sensibilidad para medir el riesgo de una cartera frente a cambios en las tasas de interés."



In-Context learning (ICL) y Few-shot prompting

- **In-Context learning** (aprendizaje en el contexto) es la habilidad que tienen los modelos de aprender a partir de ejemplos provistos en el prompt. Estos ejemplos son novedosos para la LLM, no fue entrenada con ellos.
- Estos ejemplos, se incorporan en el prompt. Esta técnica de pasar pocos ejemplos (en el orden las unidades, decenas) se conoce como **few-shot**.
- Cuando no se incluye ningún ejemplo en el prompt, se lo llama **zero-shot prompting**.

Instrucción: Clasifica el sentimiento de los siguientes comentarios de clientes como "Positivo", "Negativo" o "Neutral".

Ejemplos:

1. **Comentario:** "El paquete llegó dos días antes de lo esperado y el producto funciona de maravilla." **Sentimiento:** Positivo
2. **Comentario:** "La calidad del plástico se siente barata y se rompió al primer uso. No lo recomiendo." **Sentimiento:** Negativo
3. **Comentario:** "Es un producto promedio. Cumple con su función pero el color no es exactamente el de la foto." **Sentimiento:** Neutral

Tarea: **Comentario:** "Aunque el diseño es bonito, la batería solo dura dos horas, lo cual es muy decepcionante para el precio que pagué." **Sentimiento:**

Ejemplo de few-shot prompt

Chain of Thought prompting (CoT)

Se diseña el prompt solicitándole a la LLM que explique su razonamiento o proceso de pensamiento. Esto suele mejorar la exactitud del modelo.

Ventajas:

- suele mejorar la exactitud del modelo
- los problemas se dividen en pasos más manejables. Valioso para hacer cálculos, análisis lógicos y razonamientos multi-paso.
- puede guiar al modelo por una serie de prompts relacionados. Resulta más práctico y preciso cuando debe entender el problema primero.

Limitaciones:

- Los modelos pequeños (con pocos parámetros) ofrecen resultados CoT que no tienen sentido.
- Su efectividad también depende del tipo de tarea. Funciona mejor para sentido común, cálculos aritméticos, o razonamiento simbólico.

Chain of Thought prompting (CoT) - Ejemplo

Actúa como un analista financiero experto.

Quiero que calcules el saldo mensual disponible después de pagar los gastos, pero antes de darme la respuesta final, necesito que sigas estos pasos y me expliques tu razonamiento:

1. Identifica y suma todos los ingresos mensuales.
2. Identifica y suma todos los gastos mensuales.
3. Resta los gastos totales a los ingresos totales para obtener el saldo.
4. Calcula qué porcentaje del ingreso total representa ese saldo.

Aquí están los datos:

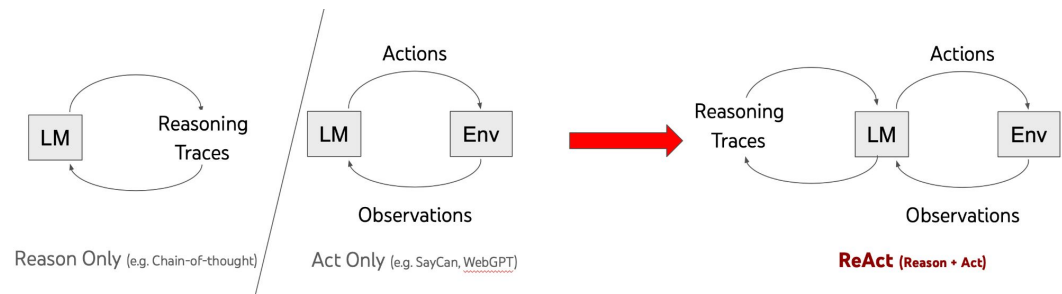
- Salario mensual: \$2,500
- Ingresos adicionales (freelance): \$300
- Alquiler: \$900
- Servicios (luz, agua, internet): \$150
- Supermercado y alimentación: \$400
- Transporte y otros gastos: \$200

Por favor, estructura tu respuesta mostrando primero la **"Cadena de Pensamiento" paso a paso** y luego el resultado final.

ReAct

En el patrón ReAct, el modelo repite un ciclo de tres fases hasta llegar a la respuesta final:

- 1. **Thought (Razonamiento):** El modelo evalúa la situación y decide qué paso lógico dar a continuación.
- 2. **Action (Acción):** El modelo invoca una herramienta externa (como una base de datos, una calculadora, una API o una búsqueda web).
- 3. **Observation (Observación):** El modelo recibe y procesa el resultado de la acción para iniciar un nuevo ciclo.



```
ReAct

Thought 1: I need to search Apple Remote and find the program
it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in
October 2005 by Apple ... originally designed to control the
Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the
Front Row media center program. I need to search Front Row
next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat
to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row
(software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

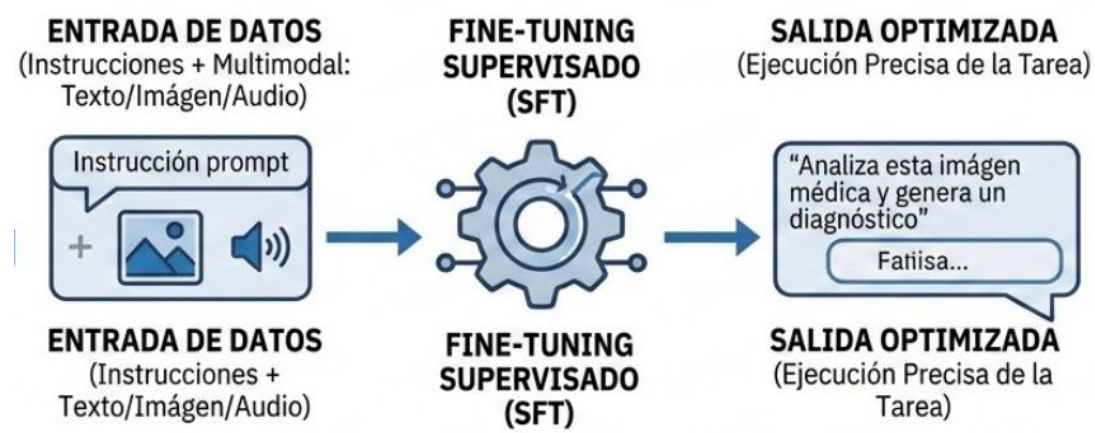
Thought 4: Front Row (software) is controlled by an Apple
Remote or the keyboard function keys. So the answer is
keyboard function keys.
Act 4: Finish[keyboard function keys]
```

<https://react-lm.github.io/>

Instruction-tuned LMMs

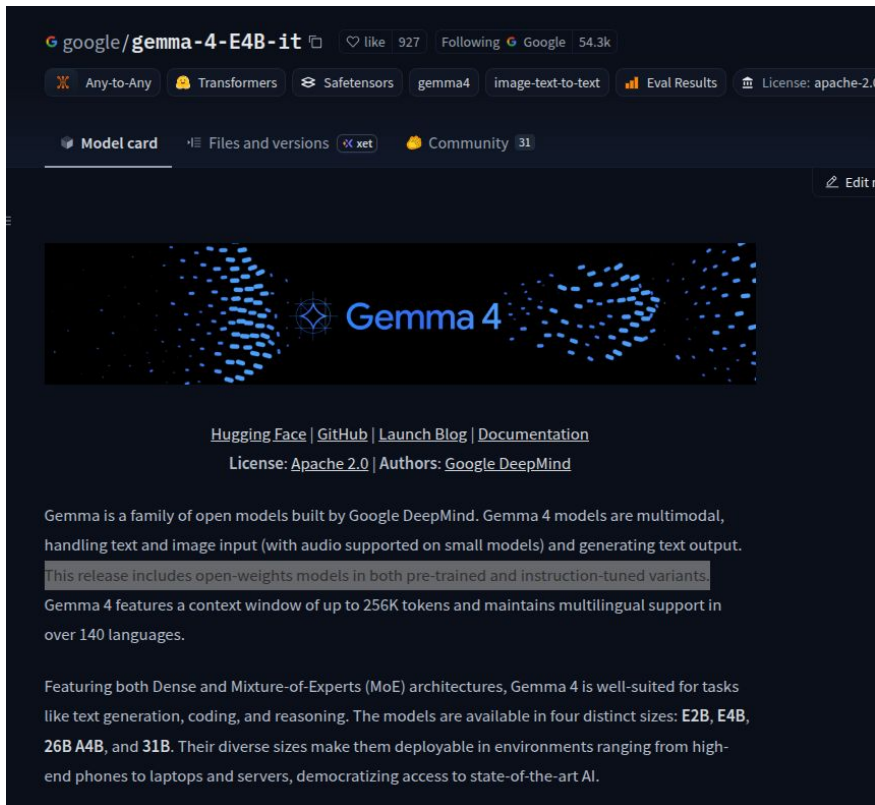
Los modelos se optimizan para seguir instrucciones en particulares. Una vez entrenado el modelo de propósito general Transformer, este es optimizado usando prompts y respuestas de ejemplo (como las ideas de few-shot) pero re-entrenando sus pesos. Este “fine-tuning” hace que el modelo ejecute mejor determinadas tareas y más alineados con las intenciones de los humanos

Hoy las mayoría de los modelos que usamos para Agentes y aplicaciones son IT.



InstructGPT: <https://arxiv.org/pdf/2203.02155>
GPT4: <https://arxiv.org/pdf/2303.08774>
<https://www.ibm.com/think/topics/instruction-tuning>

Instruction-tuned LLMs: Gemma 4 IT



google/gemma-4-E4B-it

Any-to-Any Transformers Safetensors gemma4 image-text-to-text Eval Results License: apache-2.0

Model card Files and versions xet Community 31

Gemma 4

[Hugging Face](#) | [GitHub](#) | [Launch Blog](#) | [Documentation](#)
License: [Apache 2.0](#) | Authors: [Google DeepMind](#)

Gemma is a family of open models built by Google DeepMind. Gemma 4 models are multimodal, handling text and image input (with audio supported on small models) and generating text output. This release includes open-weights models in both pre-trained and instruction-tuned variants. Gemma 4 features a context window of up to 256K tokens and maintains multilingual support in over 140 languages.

Featuring both Dense and Mixture-of-Experts (MoE) architectures, Gemma 4 is well-suited for tasks like text generation, coding, and reasoning. The models are available in four distinct sizes: **E2B**, **E4B**, **26B A4B**, and **31B**. Their diverse sizes make them deployable in environments ranging from high-end phones to laptops and servers, democratizing access to state-of-the-art AI.

Prompt

```
messages = [  
    {"role": "system", "content": "You are a helpful  
assistant."},  
    {"role": "user", "content": "Write a short joke about  
saving RAM."}]
```

<https://huggingface.co/google/gemma-4-E4B-it>

Demo

Prompt Engineering

Diferentes técnicas de Prompting

- Zero-Shot Prompting
- In-Context Learning and Few-Shot Prompting
- Role Prompting
- Chain Prompting
- Chain of Thought
- Reasoning-Act (ReAct) prompting
- Mejores prácticas

Cada modelo se comporta de manera diferente. Es un proceso iterativo.

Visual Instruction-tuned LMMs: LLaVA

Usa la idea de instruction tuning para imágenes, armando un datasets con bouding boxes (recuadros) de las imágenes para entrenar.

Context type 1: Captions

A group of people standing outside of a black vehicle with various luggage.
Luggage surrounds a vehicle in an underground parking area
People try to fit all of their luggage in an SUV.
The sport utility vehicle is parked in the public garage, being packed for a trip
Some people with luggage near a van that is transporting it.

Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], backpack: [0.384, 0.696, 0.485, 0.914], suitcase: ...<omitted>

Response type 1: conversation

Question: What type of vehicle is featured in the image?
Answer: The image features a black sport utility vehicle (SUV) ...<omitted>

Response type 2: detailed description

The image is an underground parking area with a black sport utility vehicle (SUV) parked. There are three people in the scene, with one person standing closer to the left side of the vehicle, another person in the middle, and the third person on the right side. They are all working together to pack their luggage into the SUV for a trip. ...<omitted>

Response type 3: complex reasoning

Question: What challenges do these people face?
Answer: In the image, a group of people is standing outside a black SUV in a parking area, surrounded by various pieces of luggage, including suitcases and backpacks. They are facing the challenge of fitting all their luggage into the black SUV. There are multiple suitcases and backpacks to be packed, which suggests that the group has a significant amount of belongings ...<omitted>



Table 1: One example to illustrate the instruction-following data. The top block shows the contexts such as captions and boxes used to prompt GPT, and the bottom block shows the three types of responses. Note that the visual image is not used to prompt GPT, we only show it here as a reference.

LLaVA: <https://arxiv.org/pdf/2304.08485>

Julio 2026

ECI 2026: Escuela de Ciencias Informáticas - UBA - Argentina

Prof. Leticia L Rodríguez - TEGAI

Visual input example, Extreme Ironing:	
	
User	What is unusual about this image?
LLaVA	The unusual aspect of this image is a man ironing clothes on the back of a minivan or van. This is not a typical place to perform this activity, as one would usually iron clothes in a more stationary and safe location, such as a home, using a regular ironing board. The scene depicted in the image is peculiar as it involves a makeshift ironing setup on a vehicle, which can be both unsafe and unconventional. Additionally, it is not clear how the man is able to maintain balance and stability while ironing clothes in such an unstable environment. [Start a new conversation, and clear the history]
User	What's happening in the scene?
LLaVA	The image depicts a man standing on top of a yellow SUV in a busy city street. He is holding a portable ladder, seemingly ironing clothes while standing on the vehicle. Around the scene, there are other cars, a traffic light, and a fire hydrant. Additionally, there are two other people in the vicinity, one close to the man on the car and another further away. The man on the ladder appears to be attracting attention due to his unconventional choice of ironing his clothes on top of a moving car. The city street around him is bustling with activity, adding to the unique nature of the scene.
User	What is unusual about this image?
GPT-4 [36]	The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.
User	What is unusual about this image?
BLIP-2	a man is sitting on the back of a yellow cab
User	What is unusual about this image?
OpenFlamingo	The man is drying his clothes on the hood of his car.

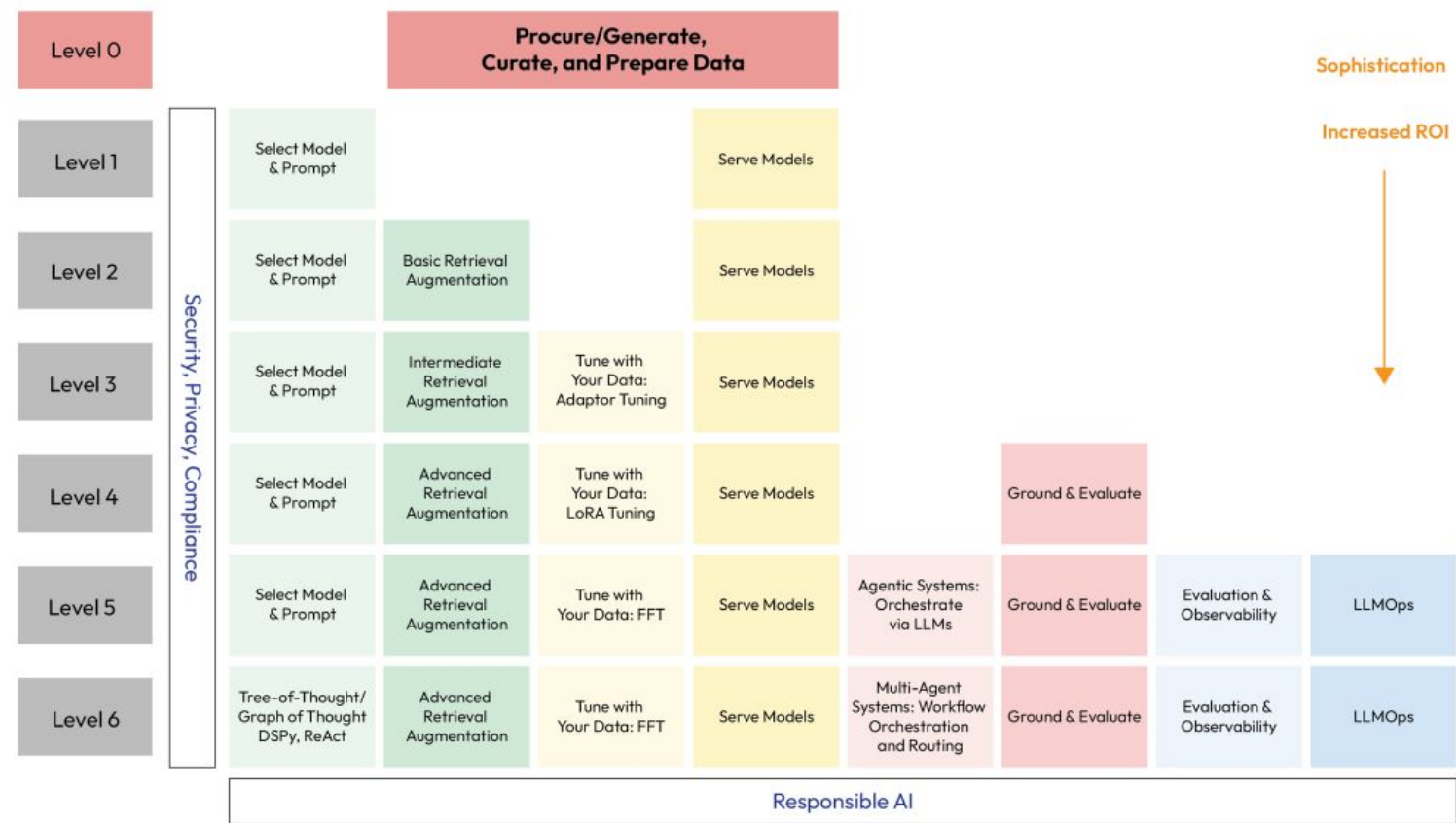
Table 3: Example prompt from GPT-4 paper [36] to compare visual reasoning and chat capabilities. Compared to BLIP-2 [28] and OpenFlamingo [5], LLaVA accurately follows the user's instructions, instead of simply describing the scene. LLaVA offers a more comprehensive response than GPT-4. Even when merely asked to describe the image, LLaVA identifies atypical aspects of the image.

Prompting: Técnicas

Más allá de las recomendaciones, hay una serie de técnicas que pueden ser usadas:

- Incluir ejemplos: few-shot
- Agregar instrucciones para indicar que la LLMs tenga que explicar su razonamiento: Chain-of-Thought (CoT)
- Framework para estructurar los prompts: ReAct (Reasoning - Act)
- Utilizar LLMs para generar prompts
- También, podemos usar prompts para juzgar la salida de otras LLMs (LLM-as-a-judge)

Madurez en GenIA: Incrementando la complejidad de las soluciones



Agentes

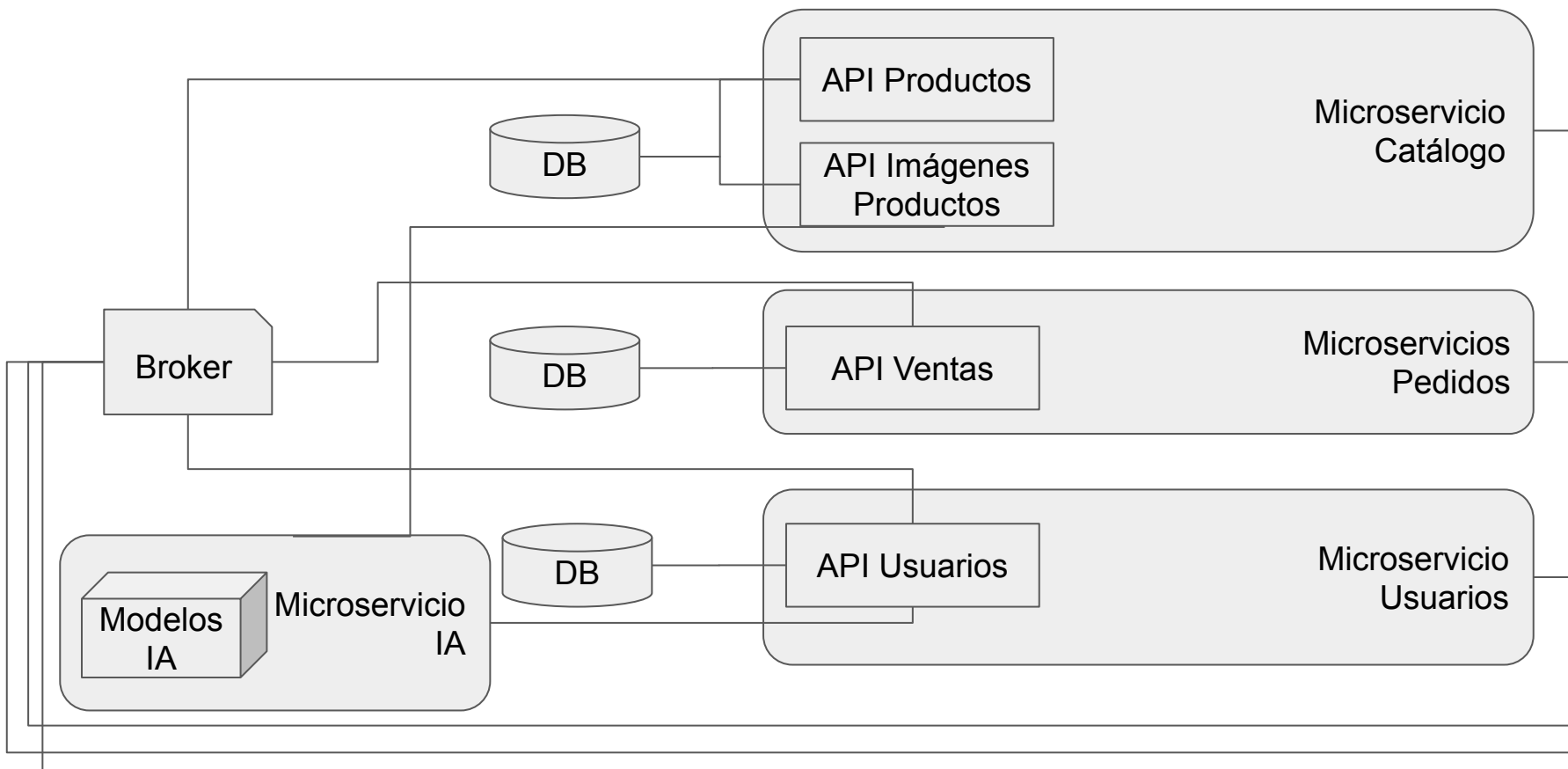
Agentes

Elementos:

- Modelos de Lenguaje ✓
- Herramientas
- Memoria
- Planificación



Microservicio y APIs



Arquitectura simplificada que muestra como se sirven modelos de IA a diferentes MS

Elementos del Agente: Herramientas

Dependiendo de la definición de agentes que tomemos, las herramientas podrían no ser necesarias. Sin embargo, para agentes que utilicen modelos de lenguajes sus capacidades estarían limitadas a la generación y búsqueda de información dentro de los conocimientos del modelo. Las herramientas sirven para aumentar las capacidades del modelo resolviendo problemas más complejos (capability extension).

Algunas herramientas incluso podrán modificar entidades externas y el entorno (write actions), en cambio, otras serán solo lecturas (knowledge augmentation).

Por ejemplo, con una herramienta podríamos:

- Actualizar una tabla en la base de datos
- Leer información de una base de datos
- Leer archivos y/o actualizar archivos
- Lanzar una búsqueda web
- Que nos permita conectarnos con el backend de alguna aplicación, por ejemplo, con los mails, documentos, drive, etc.
- Que nos permita conectarnos con una aplicación de distinta naturaleza, por ejemplo, una ticketera como Jira, Asana, etc.
- Que pueda conectar con registros médicos, o lance determinados procesos
- Que permita al modelo aumentar sus capacidades en alguna tarea que no sea bueno o sea determinística
- O usar herramientas que se conecten a nuestro sistema operativo

Estas herramientas pueden estar programadas por nosotros, ser servicios de diferentes apps o pertenecer a algún catálogo de herramientas para agentes

Herramientas Niveles

Function calling (llamada a función)	El modelo puede llamar inteligentemente a funciones acorde a lo que considere necesario entre diversas opciones que se le indica.
Tools protocols (Protocolos de Herramientas)	El agente descubre y usa herramintas externas. Por ejemplo , MCP Model Context Protocol que permite disponibilizar herramientas hosteadas
Agent-to-Agent protocols (Protocolos Agente a Agente)	Colaboración entre Agentes independientes: • Distintos frameworks • Distintas plataformas • Puede usar un protocolo estanrizado (A2A)

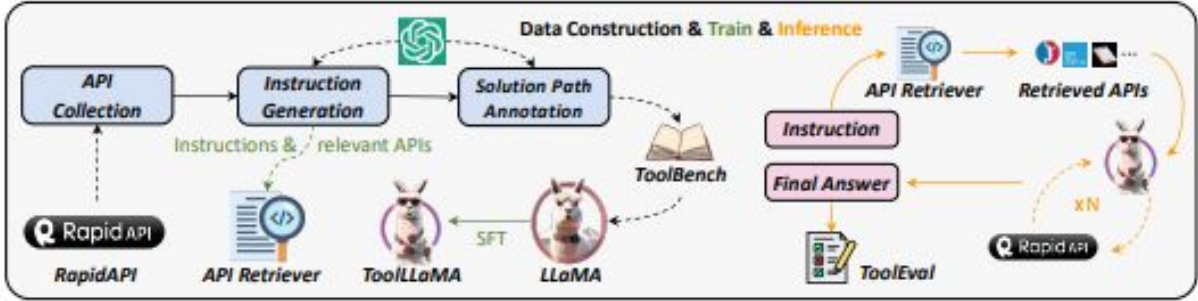


Figure 1: Three phases of constructing ToolBench and how we train our API retriever and ToolLLaMA. During inference of an instruction, the API retriever recommends relevant APIs to ToolLLaMA, which performs multiple rounds of API calls to derive the final answer. The whole reasoning process is evaluated by ToolEval.

Investigación Gorilla

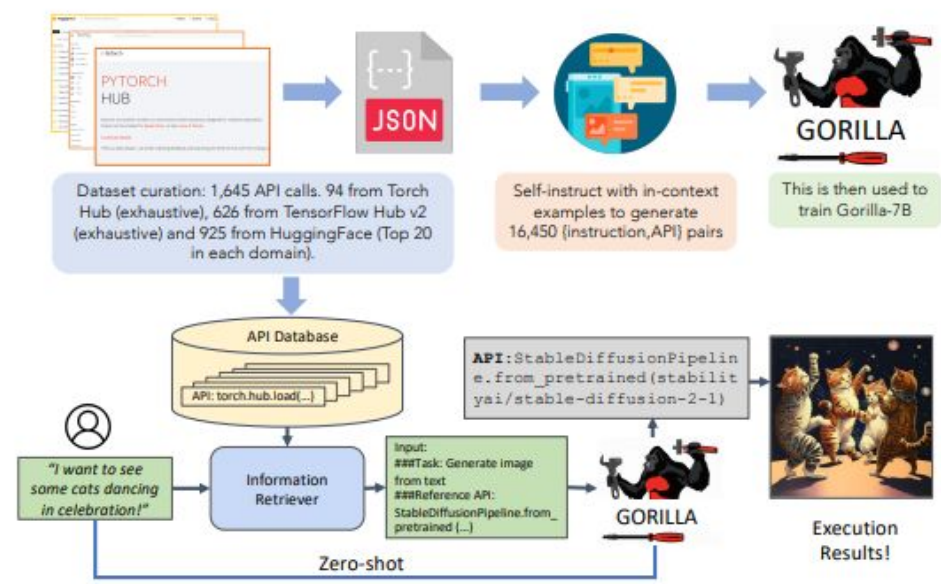


Figure 3: Gorilla: A system for enabling LLMs to interact with APIs. The upper half represents the training procedure as described in Sec 3. This is the most exhaustive API data-set for ML to the best of our knowledge. During inference (lower half), Gorilla supports two modes - with retrieval, and zero-shot. In this example, it is able to suggest the right API call for generating the image from the user's natural language query.

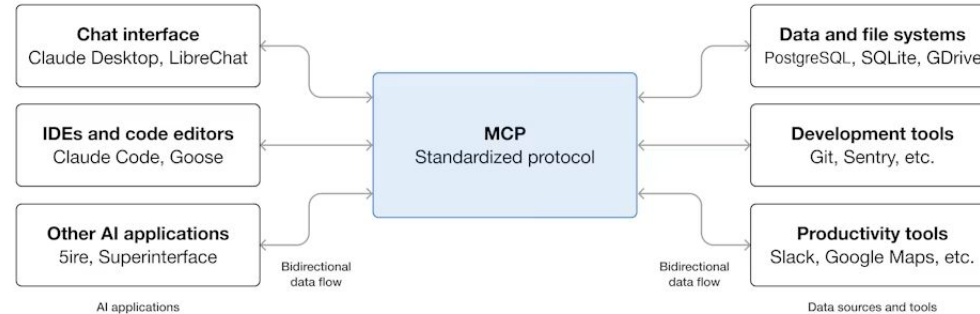
Gorilla: Large Language Model Connected with Massive APIs <https://arxiv.org/pdf/2305.15334>

MCP - Model Context Protocol

Tanto Google ADK, como otros frameworks para crear agentes permiten usar herramientas que cumplan el protocolo MCP.

MCP es un estandar abierto diseñado para que cualquier LLM pueda hacer uso de herramientas que estan creadas y hosteadas en diferentes partes.

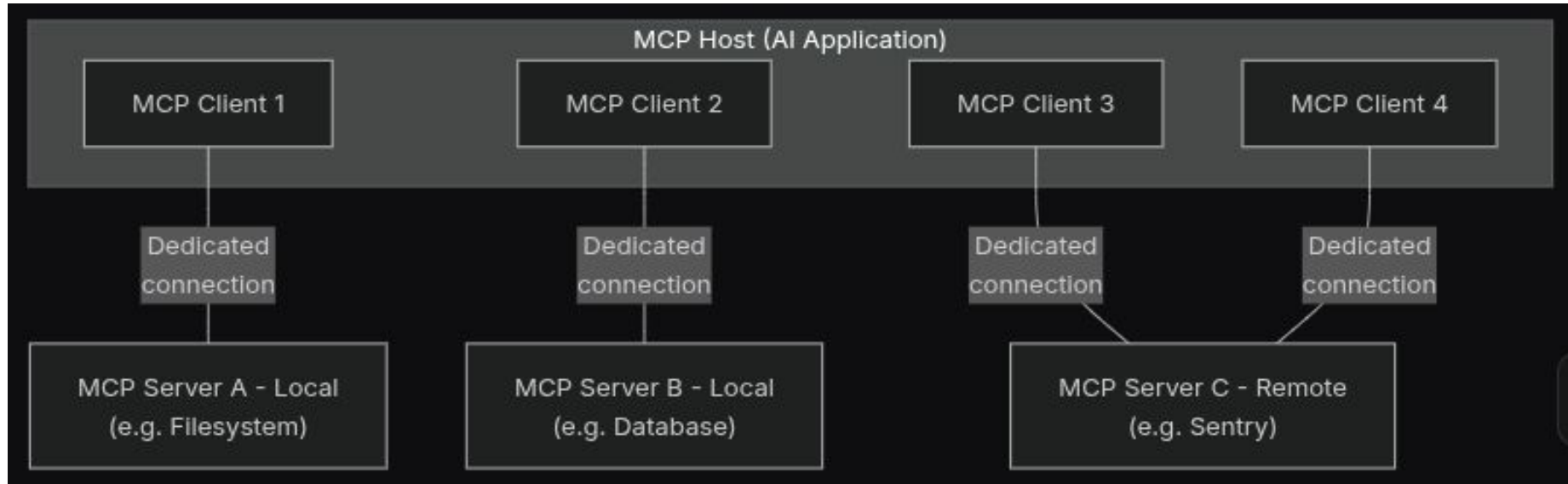
La ventaja es que permite reutilizar herramientas y estandariza la comunicación entre el agente y las herramientas externas.



MCP <https://modelcontextprotocol.io/docs/getting-started/intro>

MCP - Model Context Protocol - Arquitectura

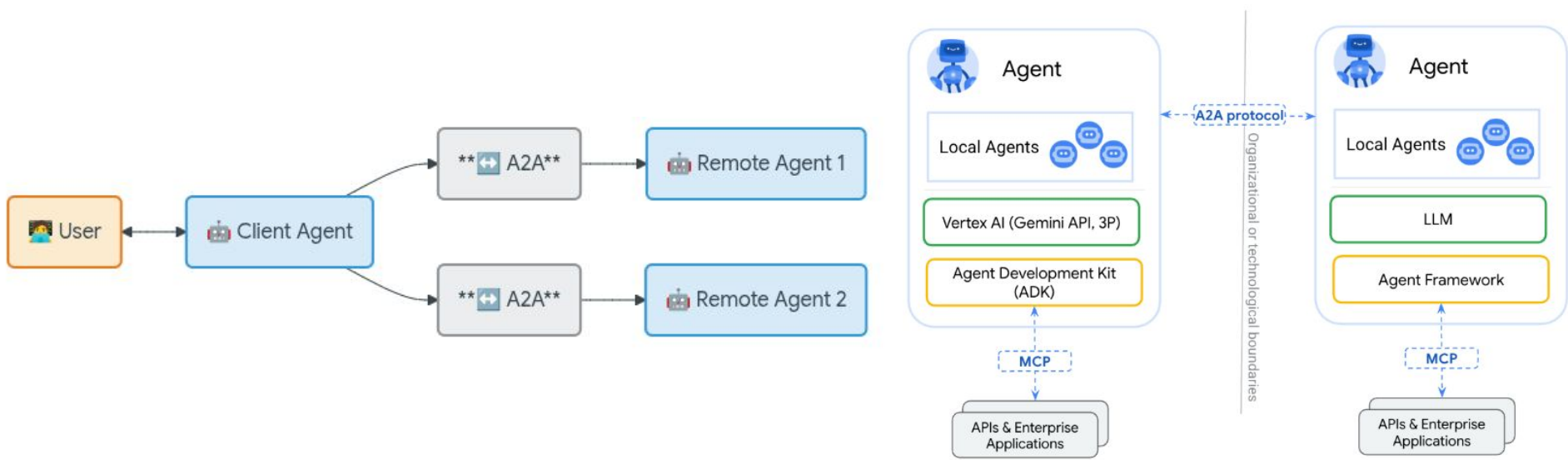
- **Host de MCP:** La aplicación de IA que utiliza uno o varios clientes de MCP, para acceder a herramientas distintas.
- **Cliente de MCP:** Un componente que mantiene una conexión con un servidor de MCP y obtiene contexto de dicho servidor para que lo utilice el host de MCP (la aplicación / Agente)
- **Servidor de MCP:** Un programa que sirve a los clientes de MCP. Da el contexto y tiene todo lo necesario para que funcione la herramienta.



MCP <https://modelcontextprotocol.io/docs/getting-started/intro>

A2A

A2A Agent to Agent - Se trata de un protocolo que permite la comunicación entre Agentes. Esto es particularmente útil si tenemos Agentes desarrollados con distinta tecnología y precisan comunicarse entre sí.

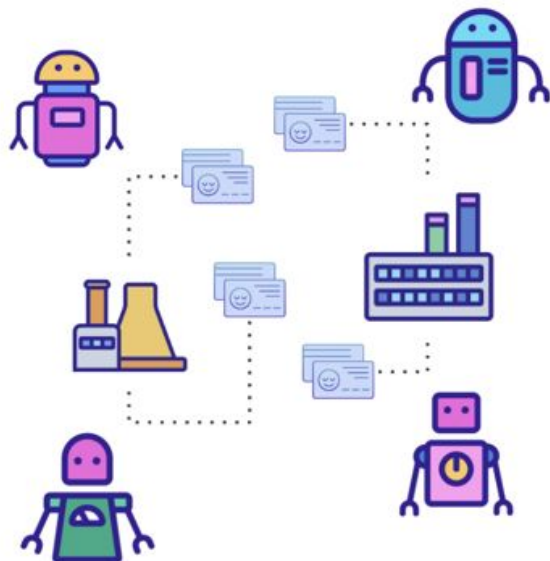


Agent-to-Agent <https://a2a-protocol.org/latest/>

A2A - Agent Card



A2A - Agent Card



Name: Agent ABC

Provider: Google (URL)

Preferred input/output: text

Capabilities:

- I can do streaming
- I can do Push Notifications

Skills:

- Skill A
 - Description
 - Input
 - Output
- Skill B
 - Description
 - Input
 - Output

Authentication:

Scheme: Bearer
Key: xxx

<https://codelabs.developers.google.com/codelabs/currency-agent#0>

Elementos del Agente: Planificación

Buscamos que el agente resuelva tareas. Para eso va a tener un gol e información. La planificación le permite definir una estrategia para resolver un gol.

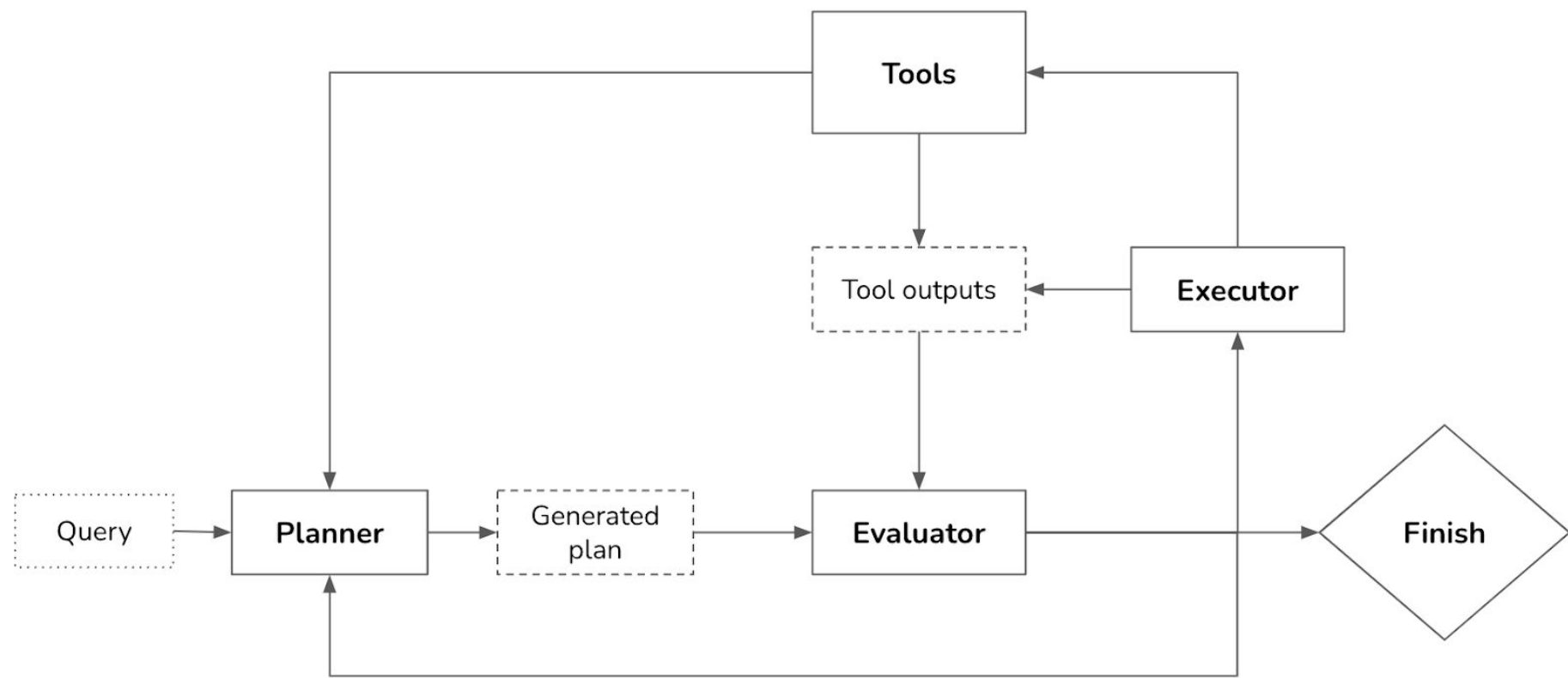
Como vimos, la publicación científica sobre chain-of-thought demostró que las LLMs completan mejor su tarea si les pedimos pensar paso a paso.

La planificación en Agentes es similar. Las tareas complejas van precisar de planificación, una especie de estrategia paso a paso que pueda ejecutar y lograr el gol propuesto.

Una planificación efectiva requiere que el modelo entienda la tarea, considere diferentes opciones para completar la tarea y luego, elija la secuencia y forma de ejecutar ese plan que considere mejor.

Chip Huyen, <https://huyenchip.com/2025/01/07/agents.html> y AI Engineering Book (2025).

Elementos del Agente: Planificación



Chip Huyen, <https://huyenchip.com/2025/01/07/agents.html> y AI Engineering Book (2025).

Elementos del Agente: Memoria

Conocimiento de un Agente

Memoria se refiere a mecanismos para permitir que el modelo almacene y utilice información. El agente usa la memoria para guardar instrucciones, ejemplos, contexto, herramientas disponibles, planes, salidas de las herramientas, reflexiones y más.

El modelo LLM tiene conocimiento interno, que retiene y fue generado sobre los datos de entrenamiento. Este conocimiento, si bien se manifiesta de forma probabilística es estático. Para cambiar lo que sabe la LLM tenemos que re-entrenarla, hacerle un grounding o un fine-tuning (ajuste fino). Accede a este conocimiento mediante un prompt (consulta) que se le hace.

Las LLMs además pueden recibir nueva información en el contexto o tener algún grounding (por ejemplo, con un RAG) para tener más información.

Memoria de un Agente

En el mundo agentico, los agentes tienen **memoria a corto plazo**. Este puede ser la memoria dentro de una sesión, interacción con un usuario. Son los mensajes anteriores y las respuestas dadas que pueden ser recordadas en la misma sesión de interacción con el usuario y que le dan contexto a la charla.

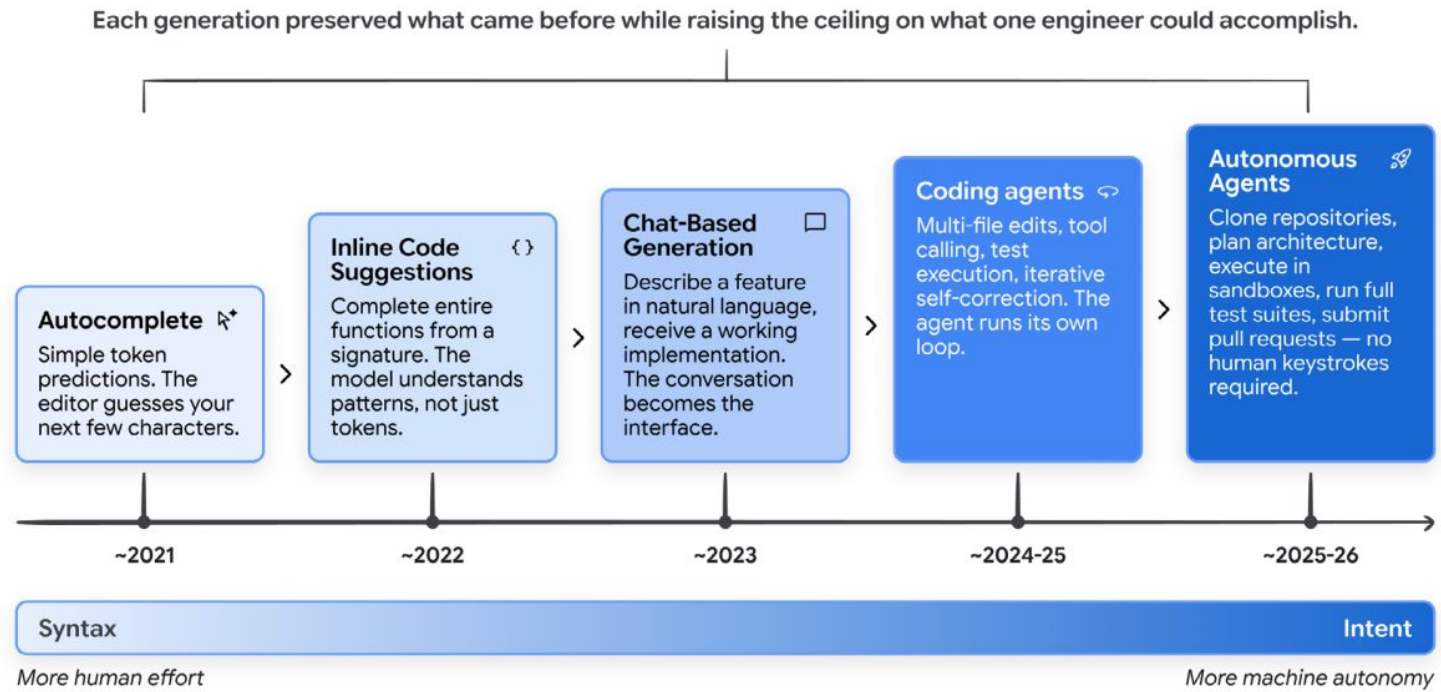
También, pueden tener **memoria a largo plazo** que se representa con datos almacenados de manera externa. Esta información persiste entre las diferentes sesiones o charlas.

<https://cloud.google.com/blog/topics/developers-practitioners/remember-this-agent-state-and-memory-with-adk>

Usando IA para Desarrollar Software

Evolución del Ciclo de Vida de Desarrollo

Con el uso de los Modelos de Lenguaje para generar y modificar código, está cambiando la forma de Desarrollar Software



Vibe Coding: El Origen



Andrej Karpathy ✓
@karpathy



Traducido del inglés [Mostrar original](#)

Hay un nuevo tipo de programación que llamo "vibe coding", donde te entregas por completo a las vibras, abrazas las exponenciales y olvidas que el código siquiera existe. Es posible porque los LLM (por ejemplo, Cursor Composer con Sonnet) se están volviendo demasiado buenos. Además, solo le hablo a Composer con SuperWhisper, así que apenas toco el teclado. Pido las cosas más tontas como "reduce el padding en la barra lateral a la mitad" porque soy demasiado perezoso para encontrarlo. Siempre "Acepto Todo", ya no leo las diffs. Cuando recibo mensajes de error, solo los copio y pego sin ningún comentario, usualmente eso lo arregla. El código crece más allá de mi comprensión habitual, tendría que leerlo realmente por un buen rato. A veces los LLM no pueden arreglar un bug, así que solo le doy la vuelta o pido cambios aleatorios hasta que desaparece. No está tan mal para proyectos desechables de fin de semana, pero sigue siendo bastante divertido. Estoy construyendo un proyecto o una webapp, pero no es realmente programar: solo veo cosas, digo cosas, ejecuto cosas y copio y pego cosas, y la mayoría funciona.

Califica esta traducción: 👍 🗨

8:17 p. m. · 2 feb. 2025 · **7,2 M** Visualizaciones

Vibe coding

[Artículo](#) [Discusión](#)

Vibe coding es una técnica de [programación](#) dependiente de [IA](#) en la que una persona describe un problema en unas pocas oraciones como [estímulo](#) para un [modelo extenso de lenguaje](#) (LLM por sus siglas en inglés) adaptado para la codificación. El LLM genera [software](#), cambiando el rol del programador de la codificación manual a la guía, prueba y refinamiento del [código fuente](#) generado por IA.^{[1](#) [2](#) [3](#)} Los defensores del *vibe coding* afirman que permite que incluso los programadores aficionados produzcan *software* sin la amplia formación y las habilidades que antes se requerían para la [ingeniería de software](#).^{[4](#)} El término fue introducido por [Andrej Karpathy](#) en febrero de 2025^{[1](#) [2](#) [4](#) [5](#)} y listado en el [Diccionario Merriam-Webster](#) el mes siguiente como un sustantivo de «jerga y tendencia».

https://en.wikipedia.org/wiki/Vibe_coding

Agentic Engineering (Ingeniería Agéntica)

Andrej Karpathy
@karpathy

Traducido del inglés [Mostrar original](#)

Un montón de gente hizo quote tweets de esto como el primer aniversario de vibe coding. Algunas reflexiones retrospectivas -

Tengo una cuenta de Twitter desde hace 17 años (omg) y todavía no puedo predecir el engagement de mis tweets básicamente en absoluto. Este fue un tuit de pensamientos al azar que lancé sin pensarlo, pero de alguna manera acuñó un nombre perfecto en el momento justo para algo que mucha gente estaba sintiendo al mismo tiempo, así que aquí estamos: vibe coding ahora se menciona en mi Wikipedia como una "contribución" memética importante e incluso su artículo es más largo. lol

Lo único que agregaría es que en ese momento, la capacidad de los LLM era lo suficientemente baja como para que la mayoría usara vibe coding solo para proyectos divertidos desechables, demos y exploraciones. Era buena diversión y casi funcionaba. Hoy (1 año después), programar vía agentes LLM se está convirtiendo cada vez más en un flujo de trabajo predeterminado para profesionales, excepto con más supervisión y escrutinio. El objetivo es reclamar el apalancamiento del uso de agentes pero sin ningún compromiso en la calidad del software. Muchas personas han intentado idear un nombre mejor para esto y diferenciarlo de vibe coding; personalmente, mi favorito actual es "agentic engineering":

- "agentic" porque el nuevo predeterminado es que no estás escribiendo el código directamente el 99% del tiempo, estás orquestando agentes que lo hacen y actuando como supervisión.
- "engineering" para enfatizar que hay un arte y una ciencia, y experiencia en ello. Es algo que puedes aprender y mejorar, con su propia profundidad de un tipo diferente.

En 2026, es probable que veamos mejoras continuas tanto en la capa de modelos como en la nueva capa de agentes. Me siento emocionado por el producto de ambos y otro año de progreso.

Home / Digital Library / Magazines / So

Call for Papers: Special Issue on Engineering Agentic Systems

IEEE Software seeks submissions for upcoming issues.

Important Dates **Submission Due:** 5 January 2026 **Expected Publication:** Sept/Oct 2026

Agentic engineering is an emerging discipline focused on the design, development, and operation of systems that exhibit goal-directed autonomy, reasoning, and continuous evolution. Foundation models (FMs), such as large language models (LLMs), have been accelerating progress in this area across academia and industry.

Agentic systems often involve multiple interacting agents, humans, and tools, requiring rigorous system-level engineering to ensure critical qualities like robustness, safety, and observability. A key design challenge in agentic engineering is the growing capability of FMs/LLMs. Developers must decide whether to rely on the FM/LLM or external tools/systems for the same functionality. These decisions can be made at various stages depending on the problem and context: during design time, development time, or event at runtime from a software engineering perspective, and at pre-training time, post-training time, test/inference time, and post-inference time from an AI perspective. Highly autonomous agentic systems also require continuous monitoring, evaluation, observability, intervention, and oversight after deployment, an emerging discipline referred to as AgentOps. Designing this post-deployment environment is also highly complex, with many interdependent design choices.

<https://www.computer.org/digital-library/magazines/so/cfp-engineering-agentic-systems>

<https://www.ibm.com/think/topics/agentic-engineering>

<https://www.langchain.com/blog/agentic-engineering-redefining-software-engineering>

Agentic Engineering (Ingeniería Agéntica)

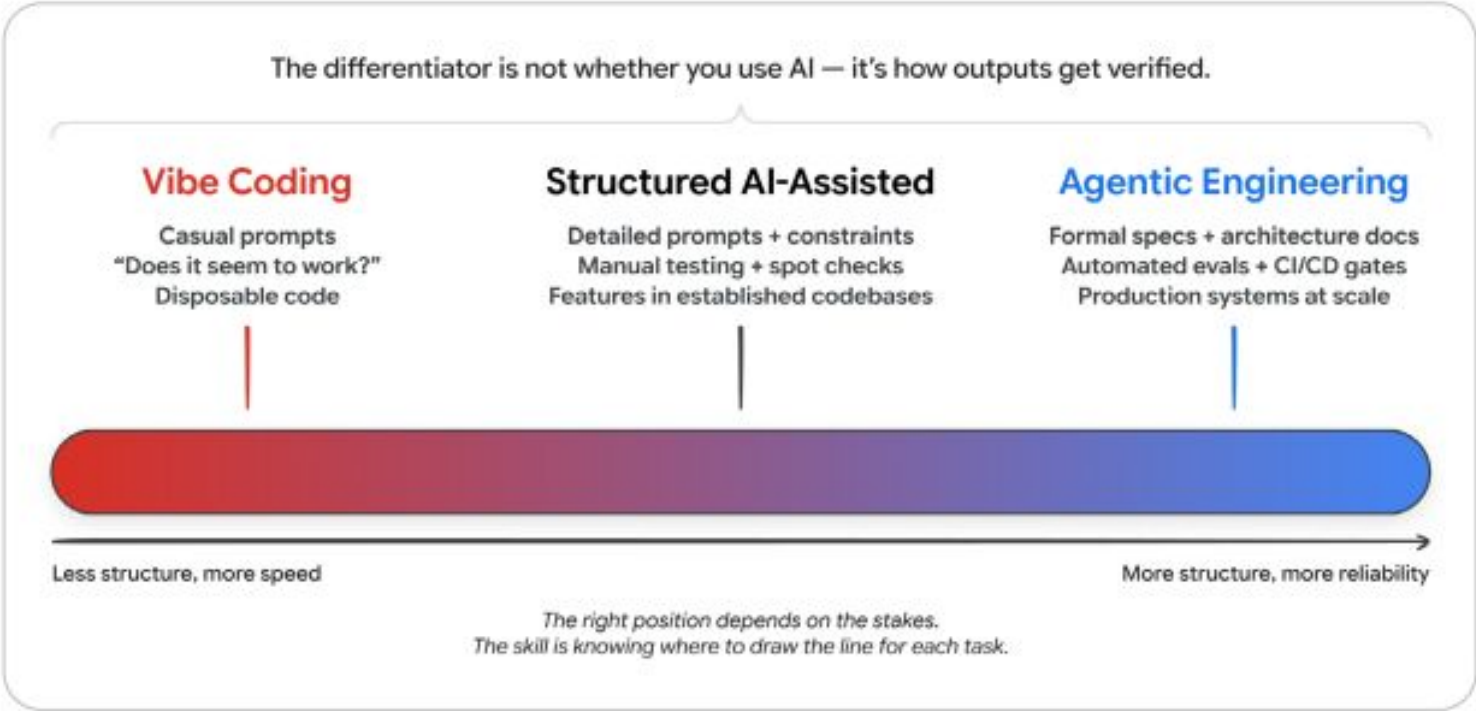


Figure 3: The Vibe Coding to Agentic Engineering Spectrum

Leticia contacto:



<https://www.linkedin.com/in/letyrodri/>